UNIT V

Tables: Rectangular tables - Jagged tables – Inverted tables - Symbol tables – Static tree tables - Dynamic tree tables - Hash tables. Files: queries - Sequential organization – Index techniques. External sorting: External storage devices – Sorting with tapes and disks.

2 MARKS

1. What are the methods available in storing sequential files?

The methods available in storing sequential files are:

- Straight merging,
- Natural merging,
- Polyphase sort,
- Distribution of Initial runs.

2. What is Symbol table? (Nov 12)

- Symbol table is a data structure it contains the information about the identifier.
- Symbol table have two field
  - Identifier name
  - Memory location

IDENTIFIER NAME	MEMORY LOCATION
A	M.L
B	M.L
C	M.L

- There are two type of symbol table
  i. Static tree table
  ii. Dynamic tree table

3. What is Hashing function?

If X is an identifier chosen at random from the identifier space, then we want the probability that f(X)=I to be 1/b for all buckets i. then a random has an equal chance of hasting into any of the b buckets. A hash function satisfying this property will be termed a uniform hash function.

Several kinds of uniform hash function are in use.

i. Mid-square method,
ii. Division method,

iii. Folding method and

iv. Digit analysis method.

4. What is meant by Mid-Square method?

A key is multiplied by itself and the address is obtained by choosing an appropriate number of bits or digits from the middle of the square. The selection of bits or digits based on the table size and also they should fit into one computer word of memory.

E.g.: consider a key, 56789 and when it is squared we get 3224990521. if the three digit address needed, then position 5to7 may chosen, given address 900.

5. What is meant by Division Method?

In this method, integer X is to divide by M and d then to use the remainder modulo M. The hash function is

$$H(x) = x \bmod M$$

Great care should be taken while choosing value for M and preferable it should be even number. By making M a large prime number the keys are spread out evenly.

6. What is meant by Folding Method?

A key is partitioned into a number of parts, each of which has the same length as the required address. The parts are then added together, ignoring the finial carry, to form an address. For e.g. ., if keys 356942781 is to be transformed into a three-digit address.

Two types:

1. Fold- shifting: 356,942 and 787 are added to yield 079.
2. Fold-boundary method; 653, 942, and 187 are added together, yielding 782.

7. What is meant by Digit Analysis Method?

A hashing function referred to as digit analysis forms addresses by selecting and shifting digits or bits of the original key. For e.g., a key 7546123 is transformed to the address 2164 by selecting digits in positions 3to6 and revising their order. Digit positions having the most uniform distributions are selected. This hashing transformation technique has been used in conjunction with static key sets.

8. What is meant by open Addressing?

Here, collisions are simply resolved by computing a sequence of hash slots. Two types of techniques:

i.Linear probing

ii.Quadratic probing.

9. What is meant by tables?

- Table is a data structure which plays a significant role in information retrieval.
- A set of n distinct records with keys K1, K2, …., Kn are stored in a file.
- If we want to find a record with a given key value, K.
- The searching time required is directly proportional to the number of number of records in the file.

10. Mention some of possible kinds of tables.

Some of possible kinds of tables are given below:

- Rectangular table
- Jagged table
- Inverted table
- Hash tables

11. What are the storage classifications?

Storage classifications:

- Volatile storage: loses contents when power is switched off
- Non-volatile storage:
  - Contents persist even when power is switched off.
  - Includes secondary and tertiary storage, as well as batter-backed up main-memory.

12. What are the Storage Hierarchy?

Storage Hierarchy:

- Primary storage: Fastest media but volatile (cache, main memory).
- Secondary storage: next level in hierarchy, non-volatile, moderately fast access time
  - Also called on-line storage
  - E.g. Flash memory, magnetic disks
- Tertiary storage: lowest level in hierarchy, non-volatile, slow access time
  - Also called off-line storage
  - E.g. Magnetic tape, optical storage

13. Write short note on Magnetic-disk.

- Data is stored on spinning disk, and read/written magnetically
- Primary medium for the long-term storage of data; typically stores entire database.
- Data must be moved from disk to main memory for access, and written back for storage
  - Much slower access than main memory (more on this later)
- direct-access – possible to read data on disk in any order, unlike magnetic tape

14. Write short note on Read-write head.

- Positioned very close to the platter surface (almost touching it).
- Reads or writes magnetically encoded information.

15. Write short notes on MTTF.

Mean time to failure (MTTF) – the average time the disk is expected to run continuously without any failure.

- Typically 3 to 5 years
- Probability of failure of new disks is quite low, corresponding to a "theoretical MTTF" of 30,000 to 1,200,000 hours for a new disk
  - E.g., an MTTF of 1,200,000 hours for a new disk means that given 1000 relatively new disks, on an average one will fail every 1200 hours

- MTTF decreases as disk ages

16. Write short notes on Optical storage.

- Non-volatile, data is read optically from a spinning disk using a laser
- CD-ROM (640 MB) and DVD (4.7 to 17 GB) most popular forms
- Write-one, read-many (WORM) optical disks used for archival storage (CD-R and DVD-R)
- Multiple write versions also available (CD-RW, DVD-RW, and DVD-RAM)
- Reads and writes are slower than with magnetic disk
- Juke-box systems, with large numbers of removable disks, a few drives, and a mechanism for automatic loading/unloading of disks available for storing large volumes of data

17. Write short notes on Magnetic Tapes.

- Non-volatile, used primarily for backup (to recover from disk failure), and for archival data.
- Sequential-access – much slower than disk.
- Very high capacity (40 to 300 gb tapes available).
- Hold large volumes of data and provide high transfer rates.
- Few GB for DAT (Digital Audio Tape) format, 10-40 GB with DLT (Digital Linear Tape) format, 100 GB+ with Ultrium format, and 330 GB with Ampex helical scan format.
- Transfer rates from few to 10s of MB/s.

18. Write short notes on Hash tables. (Nov 14)

- Hashing is differs the address or location of identifier is obtained by computing the function f(x).
- It gives address is referred as hash or address of x.
- The memory address refer to hash table (HT) hash table divided in number of buckets.
- HT (0)……HT (B-1).Each bucket can hold S records.
- Each bucket contain of S slot. Each slot can hold one record.

19. Write short notes on Floppy Disks.

- Mylar plastics usually 5 ½ or 8 inches in dia-coating magnetic material.
- 8 to 26 sector/track with 128 to 512 bytes.
- Capacity between 125 KB to 1 MB and transmission rate is 5 to 10 characters/m sec.

20. Define sequential files (Apr 12)

Sequential files have data records stored in a specific sequence. A sequentially organized file may be stored on either a serial-access or a direct-access storage medium.

21. Methods used for transaction logging:

The common method is to use transaction logging this works as follows:

- Collect records for insertion in a transaction file in their order of arrival.
- When population of the transaction file has ceased, sort the transaction file in the order of the key of the primary data file.
- Merge the two files on the basis of the key to get a new copy of the primary sequential file.

22. What do you mean by insertion in sequential files?

Records must be inserted at the place dictated by the sequents of the keys. As it is obvious direct insertion in ti the main data file would lead to frequent rebuilding of the file. This problem could be mitigated by reserving overflow areas in the file for insertions. But this leads to wastage of space and also the overflow areas may also be filled

23. What do you mean by deletion in sequential files?

Deletion is the reverse process of insertion. The space occupied by the record should be free for use. Usually deletion (like insertion) is not done immediately. The concerned record (along with a marker or `tombstone' to indicate deletion) is a transaction file. At the time of merging the corresponding data record will be dropped from the primary data file

24. What do you mean by updation in sequential files?

Updation is a combination of insertion and deletions. The record with the new values is inserted and the earlier version deleted. This is also done using transaction files

25. What do you mean by retrieval in sequential files?

User programs will often retrieve data for viewing prior to making decisions, therefore, is is vital that this data reflects the latest state of the data if the merging activity has not yet taken place.

26. What do you mean by indexed sequential files?

An index in a set of Y, address pairs. A sequential (or sorted on primary keys) file that is indexed is called an index sequential file, The index provides for random access to records, while the sequential nature of the file provides easy access to the subsequent records as well as sequential processing.

27. Define a field.

It is an elementary data item characterized by its size, length and type.

For example: Name: A character type of size 10

Age: A numeric type

28. Define a record.

It is a collection of related fields that can be treated as a unit from an applications point of view.

For example,: A university could use a student's record with the fields, university enrolment no., name, major subjects.

29. Define an index file. (Apr 13)

An index file corresponds to a data file. Its records contain a key field and a pointer to that record of the data file which has the same value of the key field.

The data stored in files is accessed by software which can be divided into the following two categories.

30. Explain hash collision (Nov 11)

Two distinct keys hashing to same index causes collision. It cannot be avoided unless have a ridiculous amount of memory.

31. Define VSAM file (Nov 11)

Virtual storage access method (VSAM) is an IBM DASD file storage access method

32. What is sequential file access? (May 14)

Sequential Access files access the contents of a file in sequence - hence the name. A files access pointer will start at the first character in the file and proceed until the last eof(). The advantage of this method of file access is that it is relatively simple.

33. What is the use of symbol table? (Nov 14)

Symbol table is an important data structure created and maintained by compilers in order to store information about the occurrence of various entities such as variable names, function names, objects, classes, interfaces, etc. **Symbol table** is **used** by both the analysis and the synthesis parts of a compiler.

34. What is latency time? (Nov 14)

It is the time required for the disk to rotate to beginning of correct sector.

35. Define external sorting.

External sorting is a term for a class of **sorting** algorithms that can handle massive amounts of data. **External sorting** is required when the data being sorted do not fit into the main memory of a computing device (usually RAM) and instead they must reside in the slower **external** memory (usually a hard drive).

36. Difference between external sorting and internal sorting.

- In internal sorting all the data to sort is stored in memory at all times while sorting is in progress.
- Shell sort, insertion sort are some of the internal sorting techniques.
- External sorting is usually applied in cases when data cannot fit into memory entirely.
- Two way Merge sort is an example of external sorting technique.

37. What are rectangular table?

Rectangular tables are also known as matrices. It is also used in all kinds of applications.

38. Write short note on jagged tables.

Jagged tables are special kind of sparse matrices such as triangular matrices, band matrices etc., In jagged tables, if elements are present then they are contiguous.

11 MARKS

1. Explain in detail about Symbol tables with its applications. (May 14)

SYMBOL TABLES-DEFINITION

- A symbol table is a set of locations containing a record for each identifier with fields for the attribute of the identifier.
- The attributes stored in a symbol table are
  - DATA TYPE: Numeric or Character
  - SCOPE: Where the program is valid
  - ARGUMENT VALUES : The argument values that is used or returned in the program
- An essential function of a compiler is to record the identifiers and the related information about its attributes types

SYMBOL TABLES-REPRESENTATION

- A typical symbol is represented as,

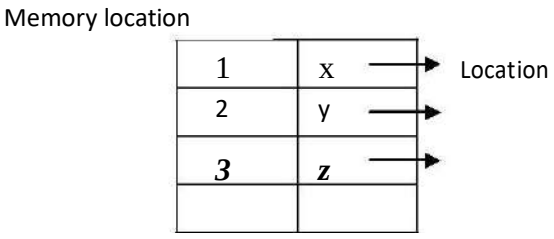


Fig.: symbol table

Where,

X, Y, Z are the variables used in the program

I-COLUMN: It contains the entry of the variables

II-COLUMN: It contains the address where the value of these variables is stored.

SYMBOL TABLE - IMPLEMENTATION

- The way to implement symbol tables are
  - ✓ Static tree table - identifiers are known in advance and no. deletions or insertions are allowed.
  - ✓ Dynamic tree table - in which identifiers are not known in advance.

STATIC TREE TABLE

- Symbol tables with property that, identifiers are known in advance and no additions on deletions are performed are called static.
- One solution is to sort the names and store them sequentially using binary search tree.
- The implementation of static tree table is carried out by binary search tree as follows,

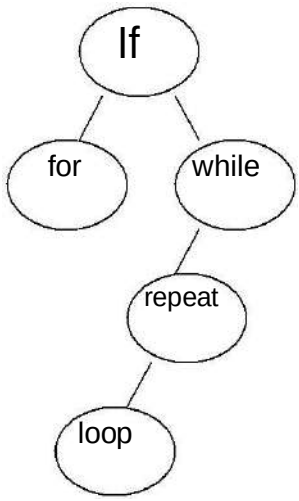
BINARY SEARCH TREE

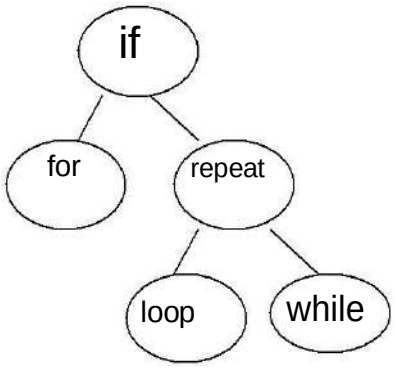
- A binary search tree T is a binary tree, either it is empty or each node in the tree contains an identifier and

- The 3 conditions for a tree to be a binary search tree are
  - ✓ All identifier in the left sub-tree of T are less (numerically and alphabetically) than the identifier in the root node
  - ✓ All identifier in the right sub-tree of T are greater (numerically and alphabetically) than the identifier in the root node T
  - ✓ The left and right sub-trees of T are also binary search trees.

Example for a BST:

- The following fig. shows two possible binary search trees for a sub set of reserved words.





SEARCH FOR A KEY IN BST:

- To determine whether an identifier X is present in a BST, X is compared with the root.
- If X is less than the identifier in the root, then the search continues in the root, the search terminates successfully, otherwise the search continues in the right sub tree.

ALGORITHM Procedure SEARCH (T,X,i)

// search the bst T for X. each node has fields LCHILD, IDENT, RCHILD. Return I = 0 if X is not in T. Otherwise, return I such that IDENT (i) = X. //

1. i <- T
2. while i ≠ 0 do
3. case
4. :X < IDENT(i) : I <- LCHILD(i) //search left sub-tree//
5. :X = IDENT(i) : return
6. :X > IDENT(i) :I <- RCHILD(i) // search right child//
7. end
8. end
9. end SEARCH

NODES IN BST:

- There are 2 types of nodes available in BST
  - ✓ Internal node
  - ✓ External node

INTERNAL NODE: The node which has its own subnodes(child)

EXTERNAL NODE: The node which does not have subnodes and remains as terminal

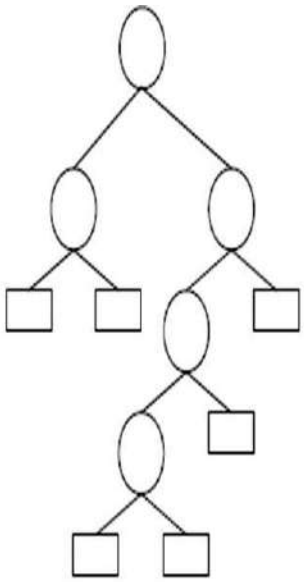
PATH LENGTH:

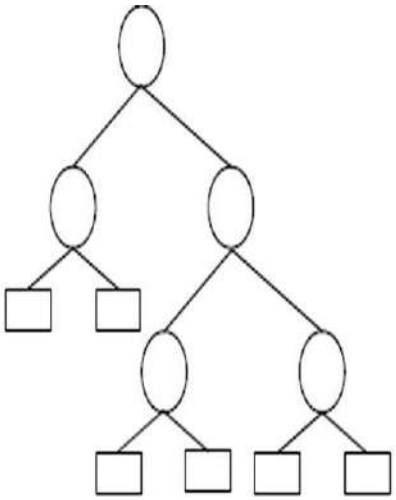
- The path length of BST is the sum of distances of the initial parent node to the last terminal of the tree
- There are two types of path length in BST ,they are
  - ✓ Internal Path Length
  - ✓ External Path Length

The calculation of the path lengths are as follows,

EXTERNAL PATH LENGTH:

- DEFINITION: The external path length of a binary tree is the sum of the lengths of the path from the root to all external nodes.
- For example consider the following diagram and its path length is calculated..



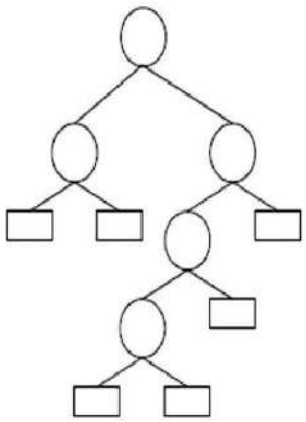


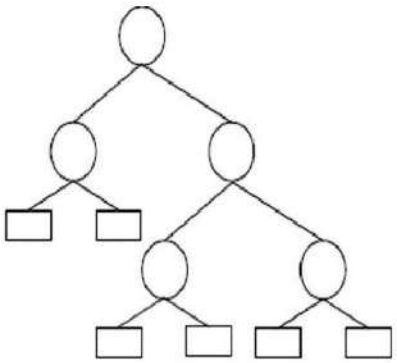
- The external path length for the above figures are,
  - ✓ FIGURE I: E=2+2+4+4+3+2=17
  - ✓ FIGURE II: E=2+2+4+4+4+4=16

Where, E is the external path length

INTERNAL PATH LENGTH

- DEFINITION: The internal path length of a binary tree is the sum of the length of the path from the root node to all internal nodes.
- The calculation of the internal path length is as follows
- Consider the following diagram





- The internal path length of the above diagram is
  - ✓ FIGURE I: I=0+1+1+2+3=7
  - ✓ FIGURE II: I=0+1+1+2+2=6

RELATION B/W INTERNAL &EXTERNAL PATH LENGTH

- The relation b/w the internal and external path length is given by the equation

E=I+2n

Where,

- E is the External path length
- I is the internal path length
- n is the no of internal nodes

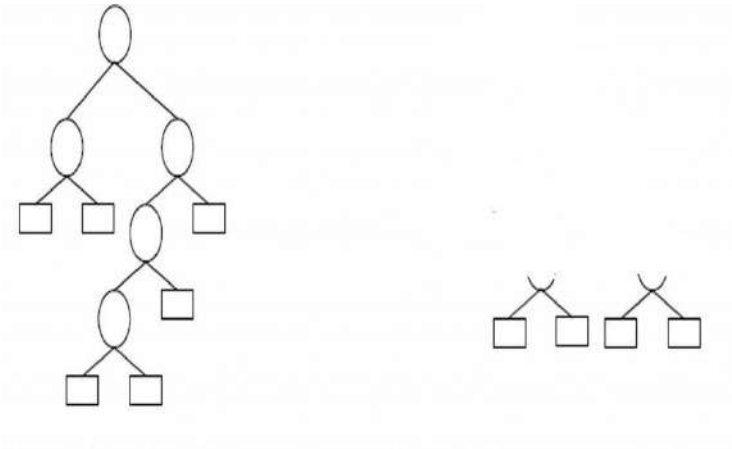
In the below picture,

E=2+2+2+3+4+4=17

I=0+1+1+2+3=7

Number of internal nodes n=5

Thus it satisfies the eqn. E=I+2n



WEIGHTED EXTERNAL PATH LENGTH:

- The weighted path length is calculated for the nodes which has its own magnitude

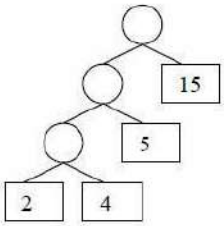
- The sum of the product of height of the node and its corresponding magnitude of all the external nodes of a tree is called as the external path length.
- The weighted external path length of a binary tree is defined to be $\sum(1 \le i \le n+1)$ [qi ki]
- where ki is the distance from the root node to the external node with weigth qi .

Consider the following diagram

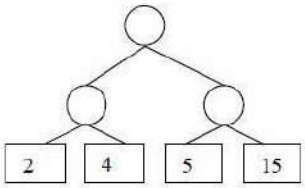
Their respective weighted external path lengths are,

->2.3+4.3+5.2+15.1 = 43

->2.2+4.2+5.2+15.2 = 52

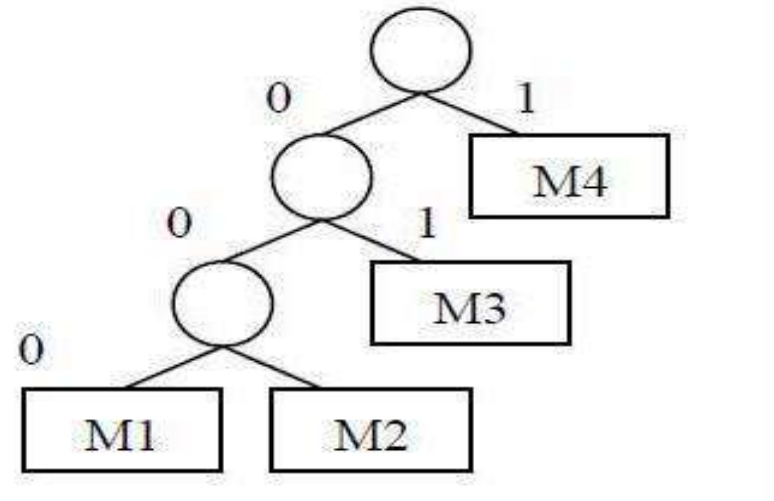


and



APPLICATION :(of binary trees with minimal weights external path length)

- Binary trees with minimal weighted external path length find application in several accesses.
  - One application is to determine an optional merge pattern using 2 – way merge sort.
  - Another application of binary trees with minimal external path length is to obtain an optional set of codes for messages M1……Mn+1.
  - Corresponding to codes 000, 0001, 01 and 1 or messages M1, M2, M3 and M4 respectively. These codes are called Huffman codes.



ALGORITHM HUFFMAN

The algorithm HUFFMAN makes use of a list L of extended binary trees. Each node in a tree has three fields (WEIGHT, LCHILD and RCHILD). Initially, all trees in L have only one node.

- For any tree in L with root node T and depth greater than 1, WEIGHT(T) is the sum of weights of all external nodes in T

PROCEDURE HUFFMAN

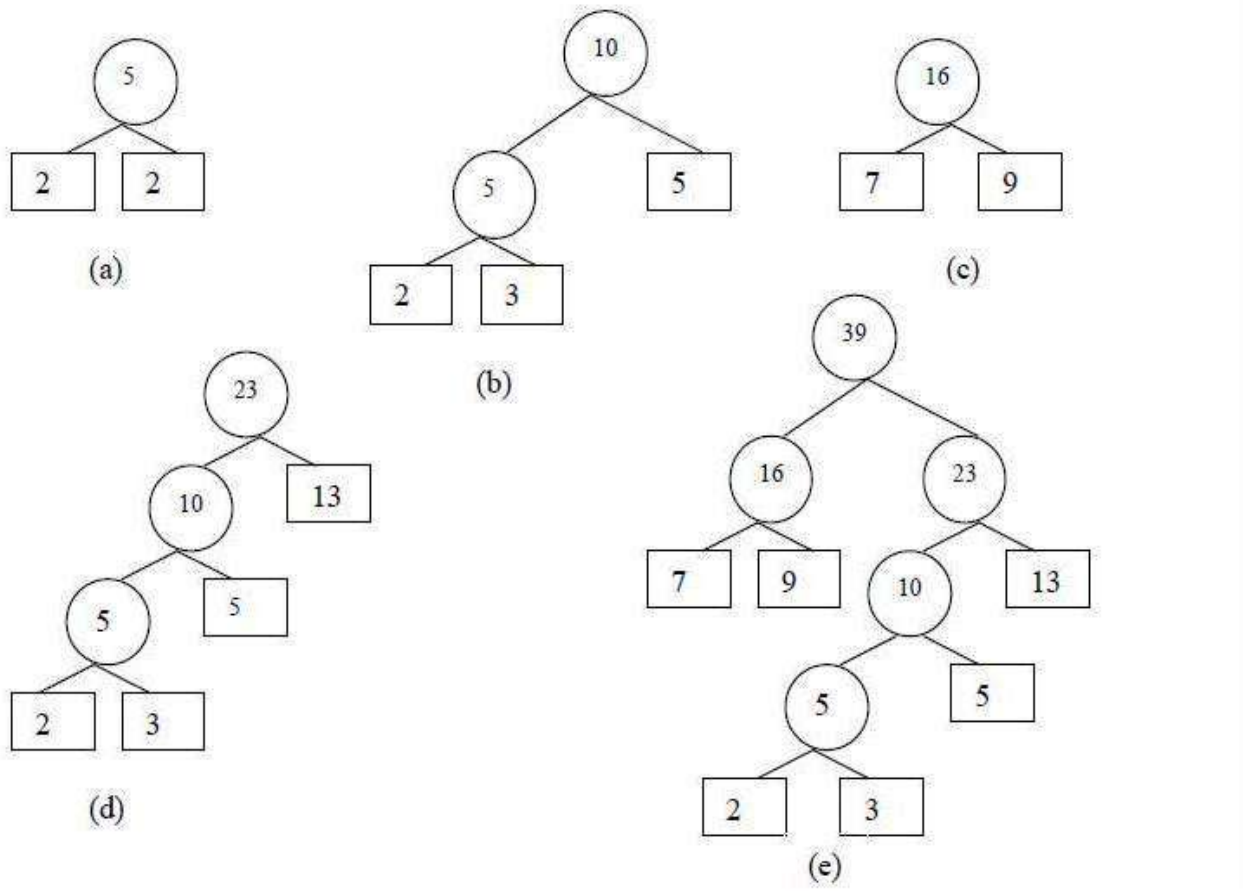
1. Procedure HUFFMAN(L,n)

// L is a list of n single node binary trees as described above //

2. for I <-1 to n-1 do //loop n-1 times//
3. call GETNODE (T) //create a new binary tree//
4. LCHILD (T) <- LEAST (L) //by combining the trees//
5. RCHILD (T) <- LEAST (L) //with two smallest weights//
6. WEGTH (T) <- WEIGHT (LCHILD (T)) + WEIGHT (RCHILD(T))
7. Call INSERT (L,T)
8. end
9. end HUFFMAN

The way that this algorithm makes is given by an example shown below

Suppose we have given the weights q1=2 , q2=3, q3=5, q4=7, q5=9 and q6=13. Then sequence of trees we would get is,

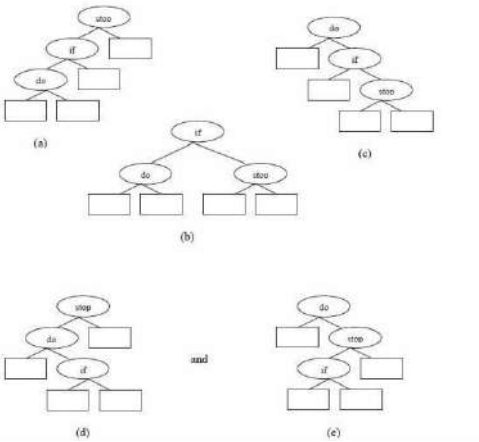


REPRESENTING SYMBOL TABLE AS A BINARY TREE:

- If the binary search tree contains the identifiers a1,a2,…..an with a1 < a2 < …< an and the probability of searching for each ai is pi.
- Then the total cost of any binary search tree is ∑(1 ≤ i ≤ n) [pi], level ai when only successful searches are made

EXAMPLE:

- The possible binary search trees for the identifier set (a1,a2,a3) = (do,if,stop) are :



2. Write in detail about Dynamic Tree Table:

- The tree table in which identifiers are not known in advance and addition and deletion are performed is a dynamic tree table.
- Dynamic table may also be maintained as binary search trees. An identifier X may be inserted into a binary search tree T by using the search algorithm to determine the failure node corresponding to X.

  This gives the position in T where the insertion is to be made.

ALGORITHM:

1. Procedure BST (X,T,j)

//search the binary search tree T for the node j such that IDENT(j) = X. If X is not already in the table then it is entered at the appropriate point. Each node has LCHILD, IDENT and RCHILD fields//

2. p <-0; j <- T //p will trail j through the tree//
3. while j ≠ 0 do
4. case
5. : X < IDENT (j) : p <- j ; j <- LCHILD(j) //search left sub-tree//
6. : X = IDENT (j) : return
7. : X > IDENT (j) : p<- j ; j <- RCHILD(j) //search right sub-tree//
8. end
9. end
10. //X is not in the tree and can be entered as a child of p//
11. call GETNODE (j) ; IDENT (j) <- X; LHCILD(j) <- 0; RCHILD(j) <- 0

12. case
13. : T = 0 ; T <- j //insert into binary tree//
14. : X < IDENT (p) : LCHILD (p) <- j
15. : else : RCHILD (p) <- j
16. end
17. end
18. end BST

DYNAMIC TREE TABLE:

The following figure shows the binary search tree obtained by entering the months JANUARY to DECEMBER in that order into an initially empty binary search tree.

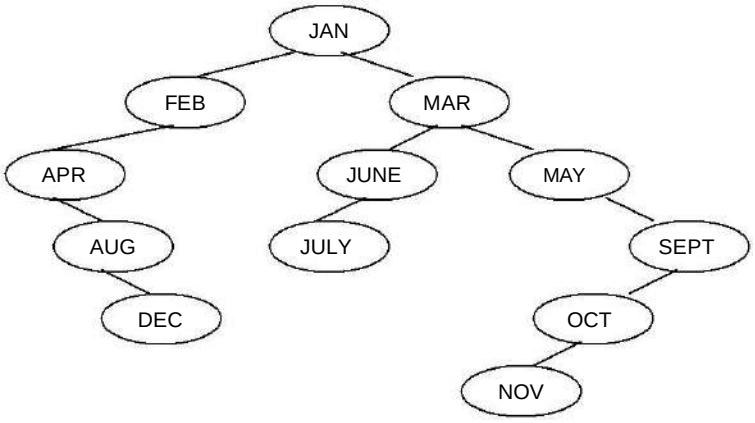


Fig.: Binary search tree obtained by entering the months

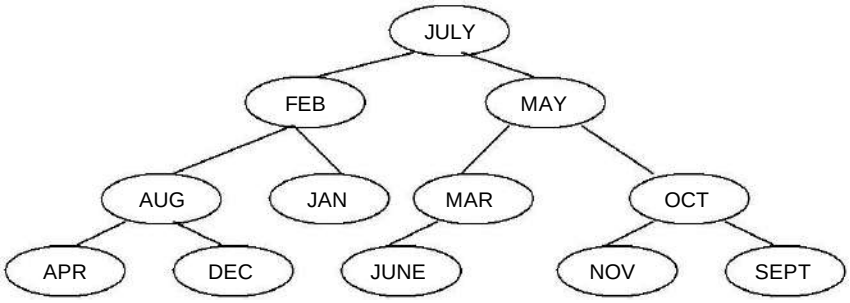
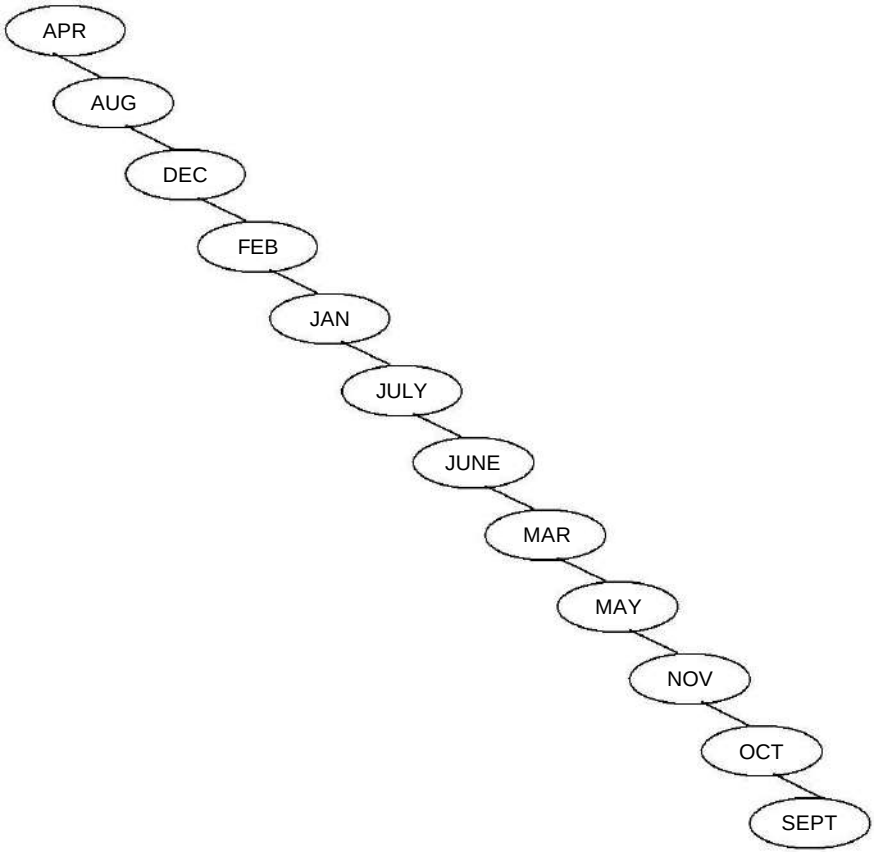


Fig.: Balanced tree for the months of the year

Degenerate binary search tree:



HEIGHT BALANCED TREE or AVL Tree or AVL Tree Rotations

If T is a nonempty binary tree with TL and TR as its left and right sub trees, then T is height balanced if TL and TR are height balanced, if it satisfies the Equation

1. |hL – hR| 1
2. Where hL and hR are the height of TL and TR respectively
3. In other words, the height of the left sub tree differs from the height of the right sub tree by no more than 1.
4. An almost height balanced tree is called an AVL tree.

BUILDING HEIGHT BALANCED:

The height balanced tree is built by using the relation

1. BF = (Height of left sub tree - Height of right sub tree)
2. If two sub-tree of same height, BF = 0
3. If right sub-tree is higher, BF = -1
4. If left sub-tree is higher ,BF=1

Consider we are ought to built an Height balanced tree with the months of a year in the order MAR,MAY,NOV,AUG,APR,JAN,DEC,JULY,FEB,JUNE, OCT,SEPT.

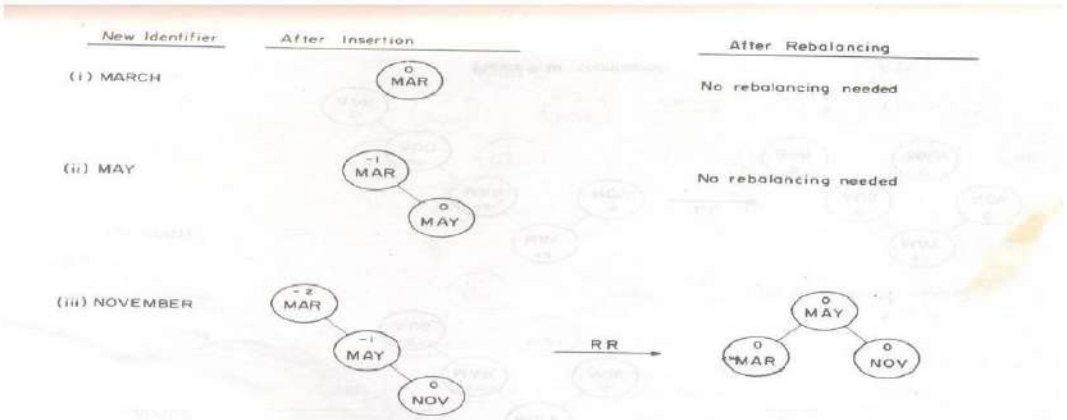


Figure 9.10 Balanced trees obtained for the months of the year.

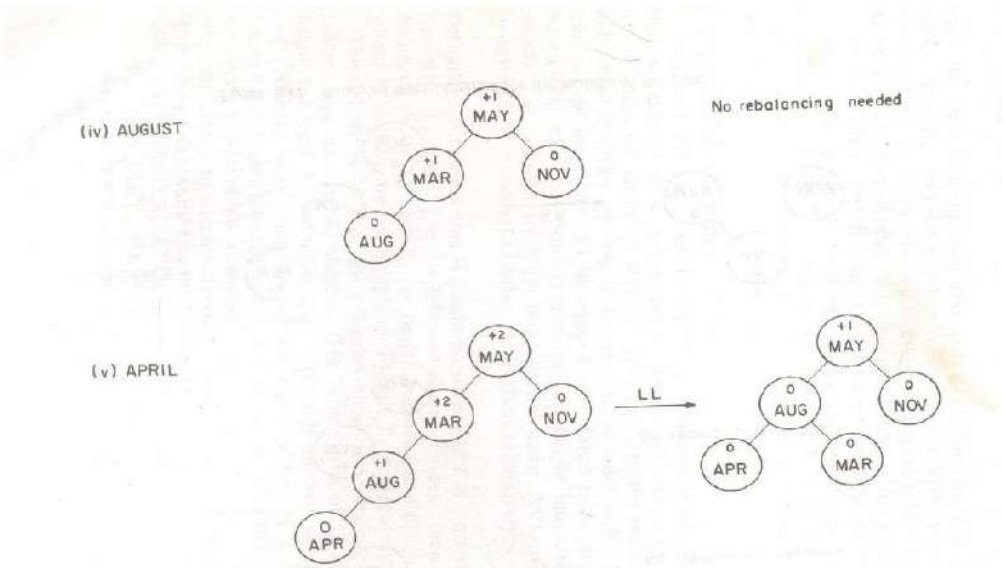


Figure 9.10 (continued)

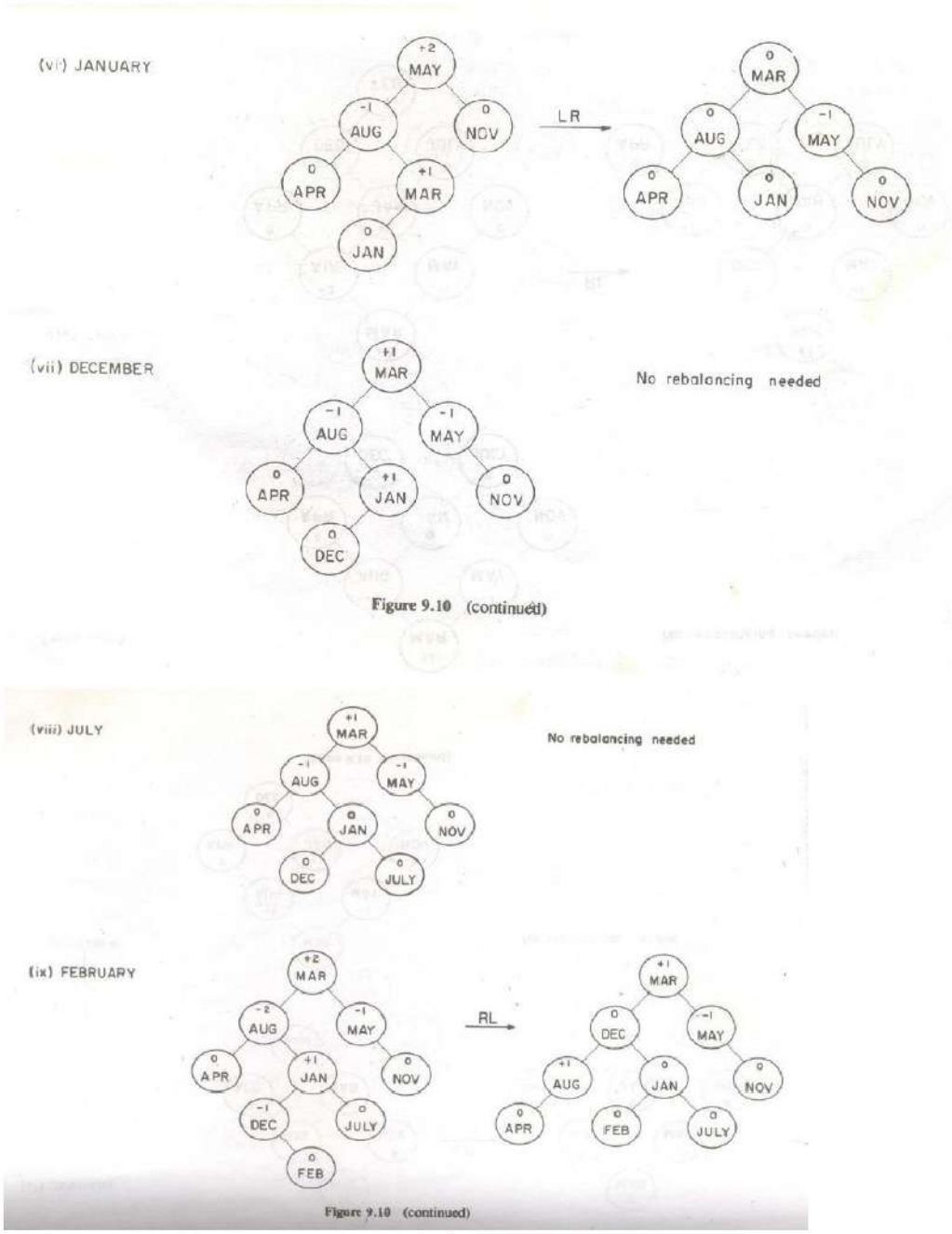


Figure 9.10 (continued)

Figure 9.10 (continued)

MULTIWAY TREE (M-TREE)

In a binary search tree, each node holds a single value and has at most two branches. Those in the left branch have less that the node value, which those in the right branch have values greater than the node value.

This can be generalized by allowing more values at each node.

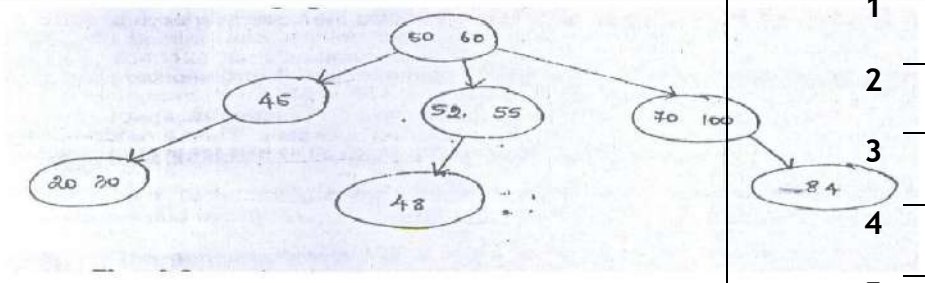
1. If we keep two values in each node, that means at most three branches, the descendants are split into three groups (maximum)
2. The leftmost descendants will have the values less than the first value in the root.
3. The middle descendant will have values between the two values in the root node.
4. The rightmost descendant will have values greater than the second value in the root node.

3. What do you mean by hashing? Explain the various hashing functions (Apr 12, Nov 14, May 15)

HASH TABLES

The best search method, binary search technique involves number of comparisons, which has a search time of $O(\log_2 n)$. Another approach is to compute the location of the desired record. The nature of this computation depends on the key set and the memory-space requirements of the desired record.

This key-to-address transformation problem is defined as a mapping or hashing function H, which maps the key space (K) into an address space (A).



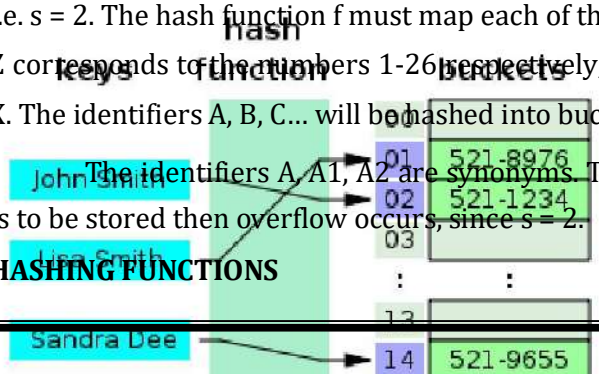
	Slot1	Slot2
1	A	A2
2	0	0
3	0	0
4	D	0
5	0	0

Fig. A small phone book as a hash table.

The hash table is partitioned into b buckets. Each bucket is capable of holding s records. Thus, a bucket is said to consist of s slots, each slot being large enough to hold 1 record.

An overflow is said to occur when a new identifier I is mapped or hashed by function f into a full bucket. Since, the key space is usually much larger than the address space; many keys will be matched to the same address. Such a many to one mapping results in collisions between records.

As an example, consider the hash table HT with b = 26 buckets, each bucket having exactly two slots, i.e. s = 2. The hash function f must map each of the possible identifiers into one of the numbers 1 – 26. Here, A-Z corresponds to the numbers 1-26 respectively, then the function f is defined by: f(X) = the first character of X. The identifiers A, B, C… will be hashed into buckets 1, 2, 3…



The identifiers A, A1, A2 are synonyms. Take for eg., if A and A1 are already stored in the bucket. If A2 is to be stored then overflow occurs, since s = 2.

HASHING FUNCTIONS

If X is an identifier chosen at random from the identifier space, then we want the probability that f(X) = I to be 1/b for all buckets i. Then a random X has an equal chance of hashing into any of the b buckets. A hash function satisfying this property will be termed a uniform hash function.

Several kinds of uniform hash functions are in use.

(i) MID-SQUARE METHOD

A key is multiplied by itself and the address is obtained by choosing an appropriate number of bits or digits from the middle of the square. The selection of bits or digits based on the table size and also they should fit into one computer word of memory.

E.g. Consider a key, 56789 and when it is squared we get 3224990521. If the three digit address is needed, then positions 5 to 7 may chosen, giving address 990.

(ii) DIVISION METHOD

In this method, integer x is to divide by M and d then to use the remainder modulo M. The hash function is
H(x) = x mod M

Great care should be taken while choosing value for M and preferably it should be an even number. By making M a large prime number the keys are spread out evenly.

(iii) FOLDING METHOD

A key is partitioned into a number of parts, each of which has the same length as the required address. The parts are then added together, ignoring the final carry, to form an address. For eg., if the key 356942781 is to be transformed into a three-digit address.

Two types:

1. Fold-shifting: 356, 942 and 781 are added to yield 079.
2. Fold-boundary method: 653, 942 and 187 are added together, yielding 782.

(iv) **DIGIT ANALYSIS METHOD**

A hashing function referred to as digit analysis forms addresses by selecting and shifting digits or bits of the original key. For eg., a key 7546123 is transformed to the address 2164 by selecting digits in positions 3 to 6 and reversing their order. Digit positions having the most uniform distributions are selected. This hashing transformation technique has been used in conjunction with static key sets.

OVERFLOW HANDLING: (or) **Collision-Resolution Technique**

Two techniques open addressing and chaining is used to detect collisions and overflows. The general objective of a collision-resolution technique is to attempt to place colliding records elsewhere in the table. This requires the investigation of a series of table positions until an empty one is found to accommodate a colliding record.

OPEN ADDRESSING

Here, collisions are simply resolved by computing a sequence of hash slots. Two types of techniques,

1) Linear Probing
2) Quadratic Probing

1. Linear Probing

Here, the function f is defined as f(I) = i. It indicates that whenever we encounter collisions, the next available cell is searched sequentially and data elements are placed accordingly.

The following figure shows a hash table with seven locations (buckets) numbered from 0 to 6. Here the divisor we use is 7. Initially, we insert 23 and its position is 2 as 23 % 7 = 2.

0	1	2	3	4	5	6
		23				

Next, we insert 50 and its position is 1 and the arrangement is as follows.

0	1	2	3	4	5	6
	50	**23**				

Then we insert 30 and its position is 2, but the bucket number 2 is already occupied by 23. So, collision has occurred. Therefore, the value 30 gets next available cell, which is 3. The orientation is as follows.

0	1	2	3	4	5	6
	50	**23**	**30**			

Similarly, when we wish to insert 38, and we face collision again. Now it is placed at the next available cell, which is 4. The arrangement of data elements is as follows.

0	1	2	3	4	5	6
	50	**23**	**30**	**38**		

Overhead in this technique is the time taken for finding the next available cell.

2. Quadratic Probing

In this case, when the collision occurs at hash address h, then this method searches the table at location h+1, h+2, and h+9. The hash function will now be defined as

$(h(x)+i^2)$ % hash size

Now let us consider a table of size 10 and index numbered from 0 to 9. Initially the table looks as follows,

0	1	2	3	4	5	6	7	8	9

When we wish to insert 23, we can easily insert at location 3 as shown below.

0	1	2	3	4	5	6	7	8	9
			23						

Next we want to insert 81 and it is easily placed at location 1.

0	1	2	3	4	5	6	7	8	9
	81		**23**						

Now we want to insert 93 and as the position 3 is already occupied, collision takes place. So, the cell with distance one apart is checked and if it is free then the new data element is placed, which is as shown below.

0	1	2	3	4	5	6	7	8	9
	81		**23**	**93**					

Now we wish to insert 113, in this case the position 3 and 4 are already occupied, so the cell with distance 4 is checked and it is found empty then the new value is placed at location 7

0	1	2	3	4	5	6	7	8	9
	81		**23**	**93**			**113**		

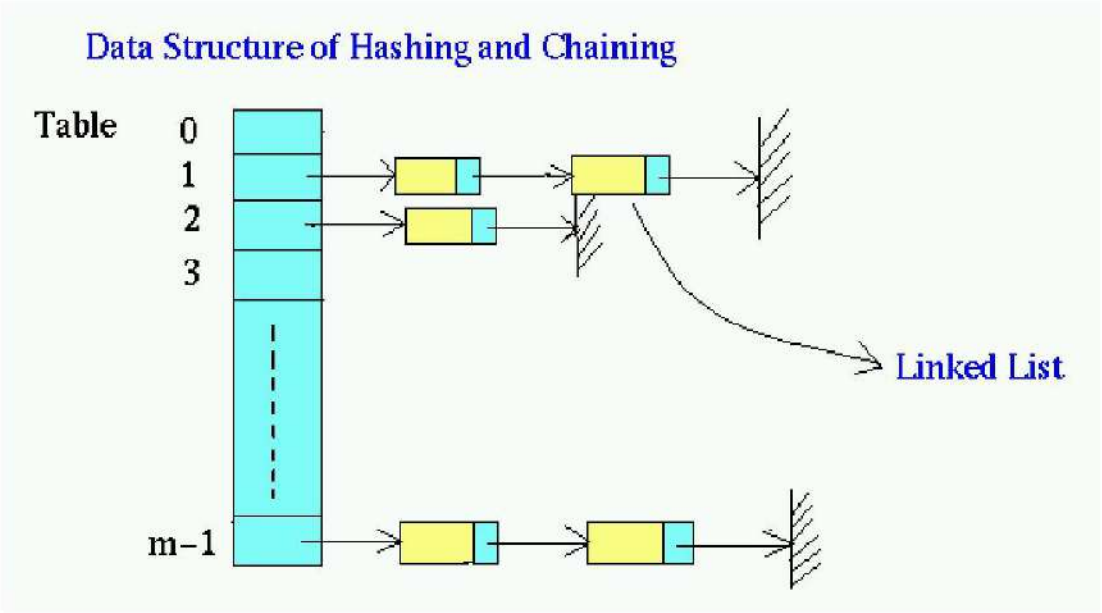
4. Explain hashing with chaining

One of the reasons Open addressing and its variations perform poorly is that searching for an identifier involves comparison of identifiers with different hash values.

Hashing with chaining is an application of linked lists and gives an approach to collision resolution. In hashing with chaining, the hash table contains linked lists of elements or pointers to elements (Figure). The lists are referred to as chains, and the technique is called chaining. This is a common technique where a straightforward implementation is desired and maximum efficiency isn't required.

Each linked list contains all the elements whose keys hash to the same index. Using chains minimizes search by dividing the set to be searched into lots of smaller pieces. There's nothing inherently wrong with linear search with small sequences; it's just that it gets slower as the sequences to be searched get longer.

In this approach to resolving collisions, each sequence of elements whose keys hash to the same value will stay relatively short, so linear search is adequate.



This Hash Table is an array [0 .. m-1] of linked_list. The table entries are called buckets or slots, and the linked lists are called chains.

x is placed in linked_list #h[x] (number of hash function).

5. Describe about direct file organization (Nov 13, Nov 14)

DIRECT FILE ORGANISATION

In order to accommodate any form of online processing which concerns the status of an account, individual customer record must be accessed directly.

THE STRUCTURE OF DIRECT FILE

- **Direct file** is also known as *Random file,* a transformation or mapping is made from the key of the record to the address of the storage location at which that record is to reside in to the file.
- The hashing algorithm used for direct files are very similar to those used for tables.
- The hashing algorithm has two components,
  - ✓ Hashing function
  - ✓ Collision resolution function
- **Hashing function:** A mapping from the key space to the address space.
- **Collision resolution technique:** Resolves conflicts that arise when more than one record key is mapped in to the same address table.
- The records in the file are stored in the buckets in which each bucket contains b record location, as opposed to just one location.
- The no. of records in the bucket is called the ***Bucket capacity***.
- For a particular record to be isolated, the bucket in which the record resides must be located, the contents of the bucket are brought in to a buffer in memory and then the desired record is extracted from the buffer.
- In a direct file, the smallest addressable unit in the bucket, this may contain many records that have been mapped to the same address.

- Hence in a direct file with a given bucket capacity, a certain no. of collisions are expected.
- When there are more colliding records for a given bucket than the bucket capacity, however, then some methods must be found for handling these over flow records.
- The term overflow handling technique is used in place of collision resolution technique.
- There has been tremendous interest in hashing techniques that enable the file to grow dynamically without requiring a significant amount of rehashing.
- These techniques are particularly applicable to direct file organization.
- Although they involve more complicated address transformation, they do result in a significantly reduced no. of collisions.

PROCESSING OF DIRECT FILES

- The processing of direct file is dependent on how the key set for record are transformed into external device addresses.
- Direct files are primarily processed directly.
- That is, the key is mapped to an address, and depending on the nature of the file transaction, a record is created , deleted , updated , or accessed at that address or possibly at some subsequent address is determined by the overflow handling technique.
- When the overflow handling is accommodated using the chaining with separate list, a pointer to a linked list of overflow records is included in each bucket.

Overflow Node

- Each overflow location in the overflow area consists of two major parts OR and KEY.
- OR - containing the address of the next location in a chain. KEY is the key of the record contained in the OR.
- A pointer to a chain of overflow records is included in the each bucket, and for the $\mathbf{i^{th}}$ bucket, this pointer is designated by PTRi.
- If there is no overflow record in the bucket, then PTRi has the value NULL. Otherwise it has the value of the address of the first record in the overflow chain for that bucket.

ALGORITHM: DIRECT_INSERT

Given a record R with key X, it is required to insert R into the direct file with n primary buckets B1,B2,……..,Bn., in which the particular bucket Bi contains m record locations Bi1,Bi2,…..Bim. If the record is resident at one location bij, then its key is denoted by kij. If no record is present, then the key field is represented with a negative number.

If an overflow condition results, then R is sorted in a location on a list of overflow location s for the primary bucket. The hashing function H is used to calculate an address.

1. [Apply hashing function]

i <- H(x)

2. [Scan the bucket indicated]

If PTRi=NULL

then repeat **for j=**1, 2, ……m.

if Kij<0

then Bij<-

R exit

3.[Put R in the overflow storage at the hed of the overflow]

OR(P)<-R

LINK(P)<-PTRi

PTRi<-P

EXIT

In step 2, the record is placed in the bucket it is hashed to if a record location is available. Otherwise, an overflow node is obtained, its address is assigned to P, record R is placed at location P, and the pointers LINK(P) and PTRi are altered so that the new node is the first in the overflow chain for the bucket i.

ALGORITHM: DIRECT_RETRIVE

Given a key **x**, it is required to retrieve the record identified by that key from the direct file with primary buckets B1,.....,Bn and the separate overflow storage.

1. [Apply hashing function]

i<-H(x)

2. [Search the bucket indicated]

Repeat fori=1, 2…

m. if x=Kij

then r<-Bij

exit

else if Kij<0

then exit

unsuccessfully. P<-PTRi

3. [Search the overflow record]

Repeat **while** true

If P=NULL

then exit unsuccessfully

if KEY (p) = X

then R<-OR (p)

exit

else P<-LINK(P)

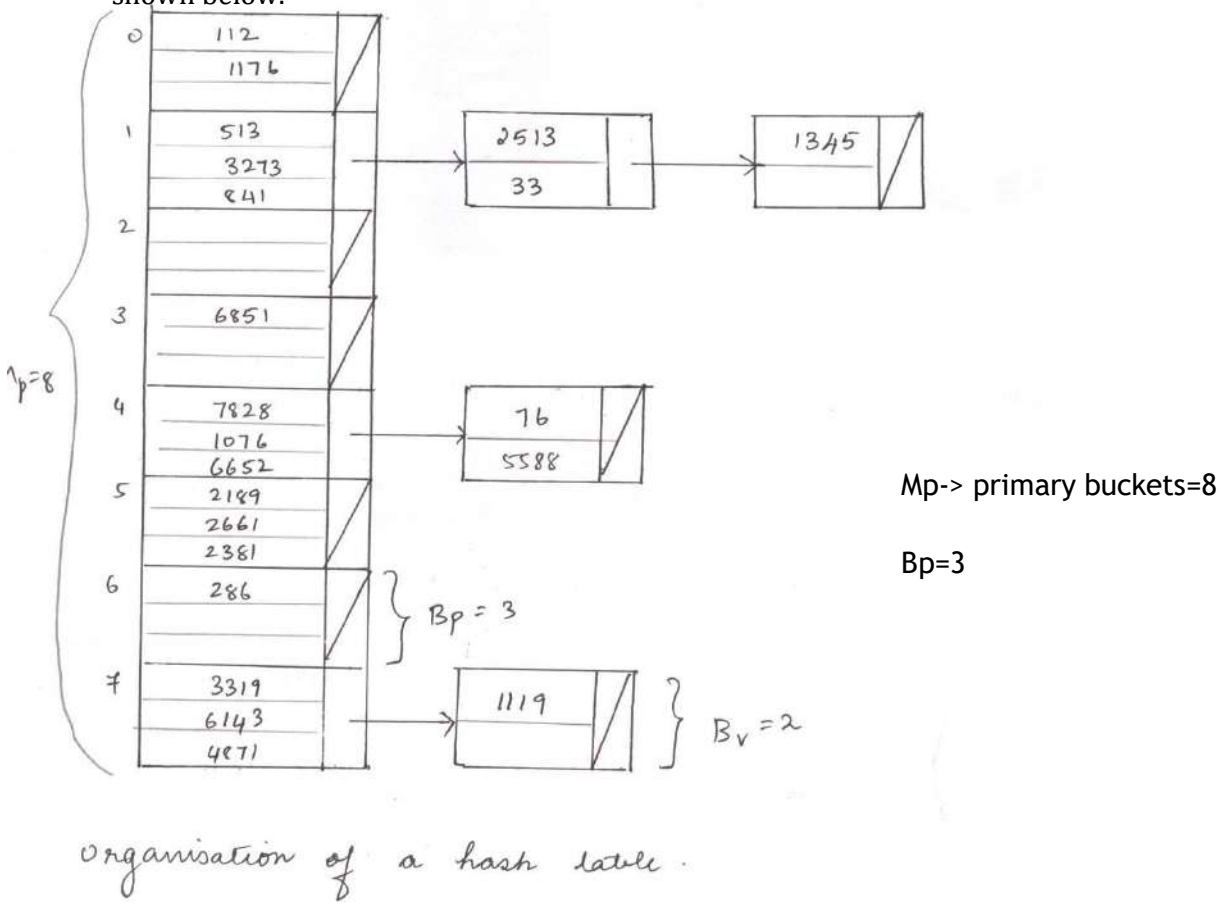
In step 2, the required record is assigned to R if it is found in the bucket Bi. If the other bucket is not full and the record is not located, then the search ends unsuccessfully. Otherwise, in step 3 each successive node of the overflow chain is examined until the record is found or the end of the linked list is encountered.

6. Explain briefly about Dynamic Hashing techniques.

DYNAMIC HASHING TECHNIQUES

Dynamic hashing technique avoids the costly rehashing of the whole file by changing the hashing function rather than moving a large number of records.

- Assume that the direct file is divided into a primary area consisting of a set of equal sized primary buckets and an overflow area consisting of equal sized overflow buckets.
- These overflow buckets are accessed using pointers. If an overflow bucked is required, an unused bucket is allocated in the same manner as a linked list element is allocated.
- Consider a direct file containing M buckets including Mp primary buckets and Mv overflow buckets as shown below.



Organisation of a hash table.

A primary bucket can hold to a maximum of Bp records and an overflow bucket can hold up to Bv records.

Hashing function uses division method to obtain a position for a key ie KEY mod TABSIZE when more than Bp keys are hashed into a location L1 an overflow bucked is allocated and associated with the primary bucket at location L.

When more than Bp+Bv keys are hashed to the location l, a second overflow bucked is added to the first

The total number of keys can be calculated by

$$N=\sum_{i=0}^{\infty} Ni$$

If No =16, N1=5, N2=1, Ni=0

N = 16+5+1+0

N =22

Effectiveness of hashing strategies are measured by load factor (α), storage utilization (β), average length of search (a<os)

Load factor $\alpha = \frac{N}{Mp\ Bp}$

$\alpha = 22/8(3)$

$= 0.9167$

$\beta = N/(mp\ bp + mv\ Bv)$

$\beta = 22/(8(3) + 4(2))$

$= 0.6375$

Length Of Search (LOS) refers to the number of buckets which must be accessed to retrieve a key for eg. LOS (1345) =3

$$ALOS = \sum_{i=0}^{\infty} \frac{(i+1) * Ni}{N}$$

$$ALOS \sum_{i=0}^{\infty} \frac{(i+1) * Ni}{N} = \frac{1(16)+2(5)+3(1)}{22}$$

ALOS = 1.318

As overflow buckets continue to be added the ALOS will rise and the access performance will deteriorate. For that, the following hashing methods are used,

1. Linear hashing.
2. Virtual hashing.

LINEAR HASHING:-

In linear hashing the table is gradually expanded by splitting the buckets in order until the table has doubted its size splitting refers to the rehashing of a bucket b and its overflows in order to distribute the keys in them among b and one other primary location.

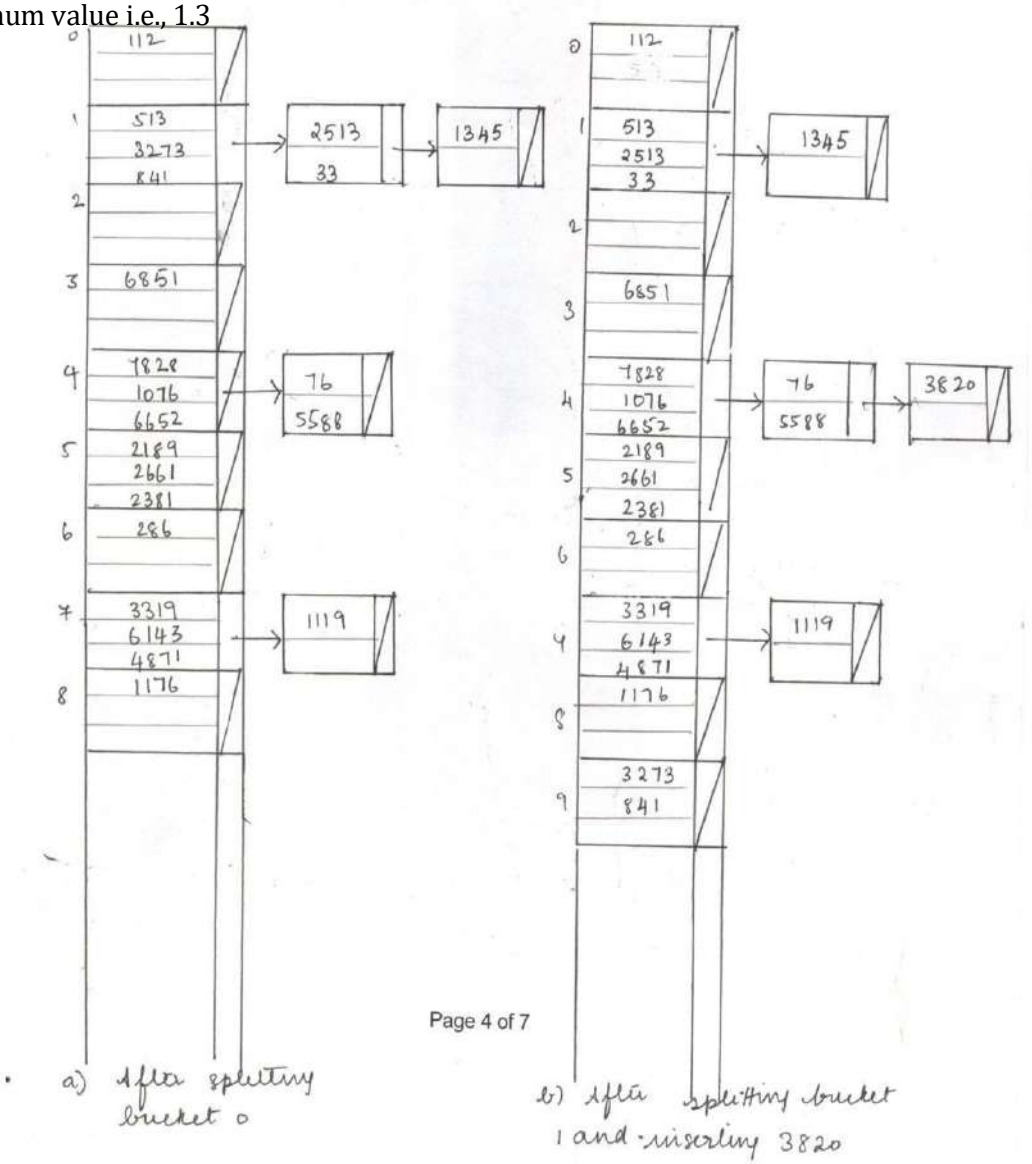
Overflow buckets may still be required, but if the hashing function selected for the rehashing is well chosen, ALOS can be reduced.

Mo-> original table size. After d doublings the size of table is 2^d Mo. Linear hashing requires the use of a series of hashing function.

Suppose that the table is doubled in size whenever an insertion at a position with a full primary bucket is to be made and the ALOS of table is already greater than 1.3.

Consider the insertion of key 3820 into the table. The hashing function tells that 3820 is to be placed at position 4, which has a full primary bucket.

Therefore ALOS =1.318 > 1.3, the table is expanded before key is added. To expand the hash table, the first bucket is rehashed using function 1+, which distributes the contents of bucket 0 between buckets 0 and 8. Buckets are split in sequence, not according to fullness. Expansion continues until the ALOS is reduced below maximum value i.e., 1.3



a) After splitting bucket 0

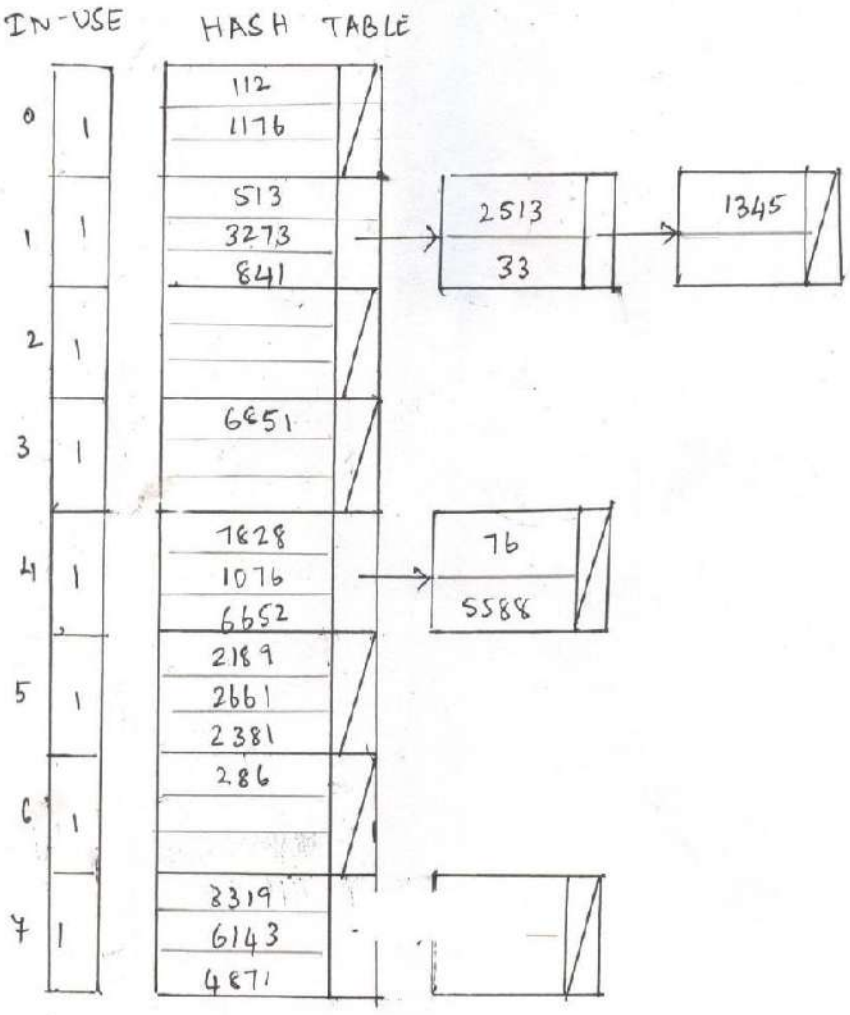
b) After splitting bucket 1 and inserting 3820

VIRTUAL HASHING

Here, the size of the table is doubled whenever the table becomes too full. A new hashing function is used with those buckets in the original table that overflow.

A separate vector of 2 Mo bit flags, tills which positions in the second half of the table are in use. As the table is increased m size, the IN-USE table grows proportionally.

Now consider the insertion of key 3820 into the table. The resulting hash table is shown in following figure.



Unexpected virtual hash table

- Hashing function Ho suggests placing key 3820 at position 4, which has a full primary bucked. Therefore ALOS=1.318 the table is doubled in size i.e. Mp is changed from 8 to 16.
- At the same time the IN_USE table is doubled in size and bits 8 through 15 of this table are set to 0.
- Rather than adding key 3820 as an overflow, all entries at 4^{th} place are rehashed using H_1. When function H_1 is used to split bucket 4, keys 6652 and 76 are moved to bucket 12. Since 12^{th} position is now in use, IN_USE [12] is set to 1. Using H_1, the suggested location for 3820 is position 12.

- To find a key, its position according to the hashing function associated with the current table size is used.
- Virtual hashing allows the size of the hash table to change without requiring the rehashing of the complete table virtual hashing preserves a low ALOS by choosing only buckets which are overflowing for splitting.

PERFORMANCE OF THE ABOVE TWO METHODS:

- Virtual hashing gives better access performance than linear hashing, but it is much less storage efficient.
- Virtual hashing splits fewer primary buckets than linear hashing. In fact, linear hashing requires fewer splits when the key set has a uniform distribution, but virtual hashing requires fewer splits when it has non-linear distribution.
- Although each of the methods has strong points, linear hashing has a significant advantage over virtual hashing. As well, linear hashing does not require the IN_USE bit table that virtual hashing needs.

7. Explain in detaim about external searching techniques. EXTERNAL SEARCHING:

Hashing technique are applied to the external files is called external searching mainly external searching are based on hashing. Other techniques that are based on tree structures such as balanced tree and tries could also be used as a basis of searching external files.

DISTRIBUTION – DEPENDENT HASHING FUNCTION:

Distribution – Dependent hashing function mainly denotes the address of the key.

In which SCK is required to find a hashing function H which maps the elements of S to address space. The required function can be obtained from discrete cumulative distribution function $Fz(x)=P(Z\leq X)$.

To find the address we have to follow digit analysis and piece-wise linear function.

DIGIT ANALYSIS:

Digit analysis is a hashing transformation which is in a sense , distribution dependent.

That digits or bits of the original key are selected and then shifted in order to form addresses.

As an example, a key 123456789 would be transform to an address 7654 if digits in positions 4 through 7 were selected and their order reversed.

For a given key set, the same position of key and the same rearrangement pattern must be used consistently.

A PIECE-WISE LINEAR FUNCTION:

- It is the second dependent distribution hashing function.
- A key space consist of integers in interval (a,d) this interval is divided into j equal subintervals of length L, ie L=(d-a)/j.
- A interval location of a given key x is found by the formula.

i=1+[(x-a)/L]

- Ni, Gi are frequency and cumulative frequency respectively of interval Ii. Using Ni and Gi, the equations

Pi(x)=(Gi+((x-a)/L-i)Ni)/N

- The required hashing function for a key x on interval Ii is,

Hi(x)=[mPi(x)],1≤i≤j for an address space of size m.

The following algorithm illustrates how the piece-wise linear function for indirect addressing is calculated.

ALGORITHM:

Piece-wise. Given j,a,d,m and n as previously defined and a key set { x1,x2,….xn}. it is required to calculate interval length L and frequencies and cumulative frequencies Ni,Gi, 1≤i≤j, for piece-wise linear function.

1. [Initialize array N to zero]

Repeat for i=1,2,……j

Ni<-0

2. [Determine interval length and interval frequency]

L<-(d-a)/j

Repeat for k=1,2,……n

I<- 1+[(Xk-a)/L]

Ni<- Ni+1

3. [Calculate interval cumulative frequencies]

G1<- N1

Repeat for i=2,3,…….j

Gi-< Gi-1+ Ni

4 [Finished]

Exit.

From the given parameters, the following assignment statement can be used to calculate an address H from a key X in interval (a,d)

i<-1+[(x-a)/L]

if Gi≠0

then H(x)=[m(Gi+((x-a)/L-i)Ni)/n]

else H(x)<- 1

ALGORITHM MULTIPLE FREQUENCY:

Let Nik be number of keys in interval Ii of the Kth key set, let Gik be the number of keys less than a+iL in same key set K. It is required to calculate H, the value of function H(x) for a key x from initial key using q frequency distribution mapping.

1. [Initialize auxillary variable]

Y<- x.

2. [Transform through key sets]

Repeat for k=1,2,……..q

i<-1+[(y-a)/L]

F<-(Gik +((y-a)/L-i)Nik)/n

If k<q

Then y<-[(d-a)F+a]

3. [Calculate an address]

If F≠0 then

H<-[mF]

Else H<- 1

4. [Finished]

Exit.

Range of key space (a,d) is initially divided into 10 intervals of length (d-a)/10.

Ni denotes frequency of ith interval. If this interval is not split then Gi has a value which represents a cumulative frequency for that interval.

If Gi has a negative value, then the absolute value of Gi gives the location of first of two consecutive pairs of array elements.

ALGORITHM INTERVAL SPLITTING:

Given L,a,m,n,p are previously defined and array N and g. It is required to calculate an address H in {1,2,…….m} from the keyx.

1 [calculate initial interval number]

R<-i<-+[(x-a)/L]

2 [Calculate interval number and array index if interval or subinterval is split]

Repeat for k= 1,2,……p-1 while

Gi<0 r<-1+[(x-a)/(L/2k)]

i<- -Gi – (r mod 2)+1

3 [calculate address]

If Gi≠0

Then H<- [m(Gi+((x-a)/(L/2k-1)-r)Ni)/N]

Else H<- 1

4 [finished]

Exit

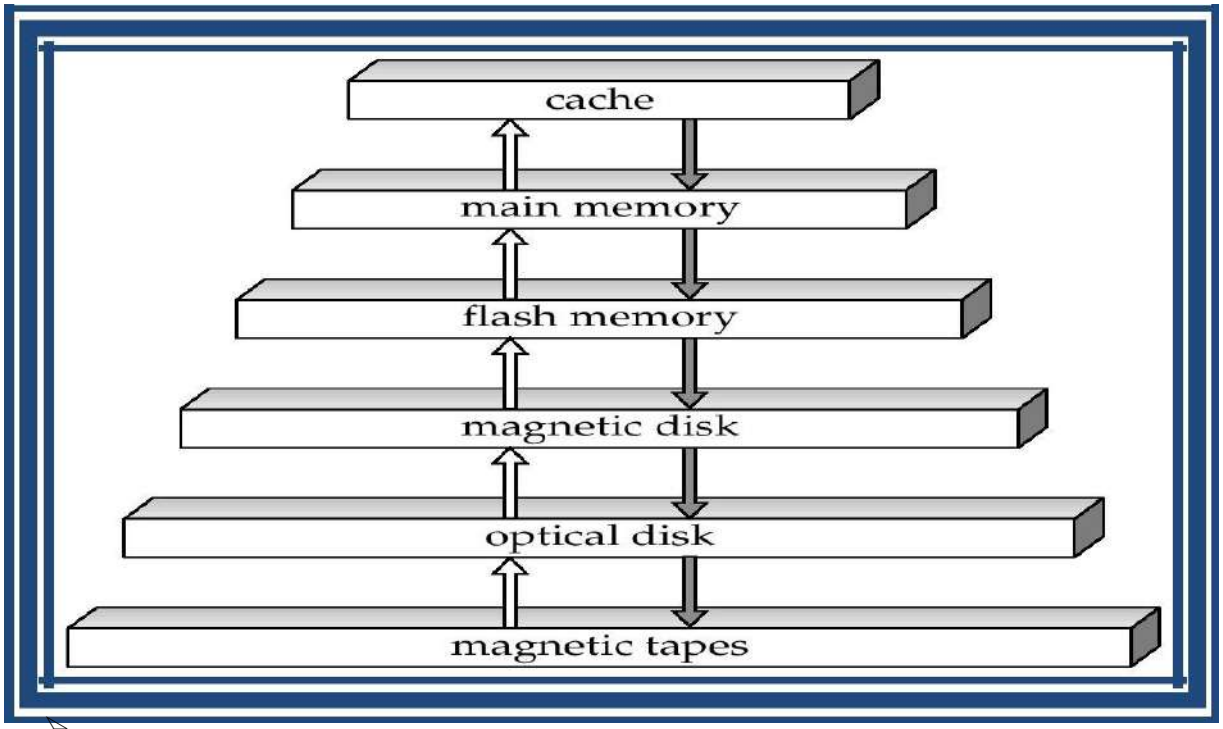
8. Discuss about external storage devices. (Nov 11)

. External Storage Devices

- Speed with which data can be accessed
- Cost per unit of data
- Reliability
  - data loss on power failure or system crash
  - physical failure of the storage device

Storage classification

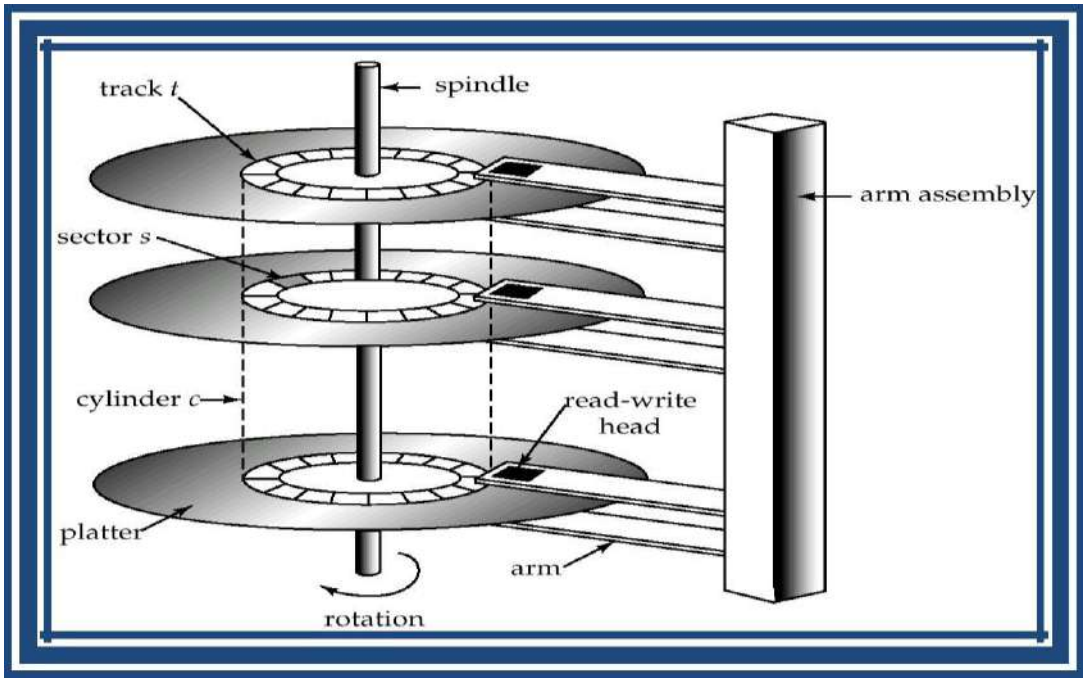
- **volatile storage:** loses contents when power is switched off
- **non-volatile storage**:
  - Contents persist even when power is switched off.
  - Includes secondary and tertiary storage, as well as batter-backed up main-memory.
- **Storage Hierarchy**



- **Primary storage:** Fastest media but volatile (cache, main memory).
- **secondary storage:** next level in hierarchy, non-volatile, moderately fast access time
  - also called **on-line storage**
  - E.g. flash memory, magnetic disks
- **tertiary storage:** lowest level in hierarchy, non-volatile, slow access time
  - also called **off-line storage**
  - E.g. magnetic tape, optical storage
- **Magnetic-disk**
  - Data is stored on spinning disk, and read/written magnetically

- Primary medium for the long-term storage of data; typically stores entire database.
- Data must be moved from disk to main memory for access, and written back for storage
  - Much slower access than main memory (more on this later)
- **direct-access** – possible to read data on disk in any order, unlike magnetic tape
- Hard disks vs floppy disks
- Capacities range up to roughly 100 GB currently
  - Much larger capacity and cost/byte than main memory/flash memory
  - Growing constantly and rapidly with technology improvements (factor of 2 to 3 every 2 years)
- Survives power failures and system crashes
  - disk failure can destroy data, but is very rare

Magnetic Hard Disk Mechanism



NOTE: Diagram is schematic, and simplifies the structure of actual disk drives

- **Read-write head**
  - Positioned very close to the platter surface (almost touching it)
  - Reads or writes magnetically encoded information.
- Surface of platter divided into circular **tracks**
  - Over 16,000 tracks per platter on typical hard disks
- Each track is divided into **sectors.**
  - A sector is the smallest unit of data that can be read or written.
  - Sector size typically 512 bytes
  - Typical sectors per track: 200 (on inner tracks) to 400 (on outer tracks)

- To read/write a sector
  - disk arm swings to position head on right track
  - platter spins continually; data is read/written as sector passes under head
- Head-disk assemblies
  - multiple disk platters on a single spindle (typically 2 to 4)
  - One head per platter, mounted on a common arm.
- **Cylinder** *i* consists of i^{th} track of all the platters
- **Performance Measures of Disks**
- **Access time** – the time it takes from when a read or write request is issued to when data transfer begins. Consists of:
  - **Seek time** – time it takes to reposition the arm over the correct track.
    - Average seek time is 1/2 the worst case seek time.
      - Would be 1/3 if all tracks had the same number of sectors, and we ignore the time to start and stop arm movement
    - 4 to 10 milliseconds on typical disks
  - **Rotational latency** – time it takes for the sector to be accessed to appear under the head.
    - Average latency is 1/2 of the worst case latency.
    - 4 to 11 milliseconds on typical disks (5400 to 15000 r.p.m.)
- **Data-transfer rate** – the rate at which data can be retrieved from or stored to the disk.
  - 4 to 8 MB per second is typical
  - Multiple disks may share a controller, so rate that controller can handle is also important
    - E.g. ATA-5: 66 MB/second, SCSI-3: 40 MB/s
    - Fiber Channel: 256 MB/s
- **Mean time to failure (MTTF)** – the average time the disk is expected to run continuously without any failure.
  - Typically 3 to 5 years
  - Probability of failure of new disks is quite low, corresponding to a "theoretical MTTF" of 30,000 to 1,200,000 hours for a new disk
    - E.g., an MTTF of 1,200,000 hours for a new disk means that given 1000 relatively new disks, on an average one will fail every 1200 hours
  - MTTF decreases as disk ages
- **Optical storage**
  - non-volatile, data is read optically from a spinning disk using a laser
  - CD-ROM (640 MB) and DVD (4.7 to 17 GB) most popular forms
  - Write-one, read-many (WORM) optical disks used for archival storage (CD-R and DVD-R)
  - Multiple write versions also available (CD-RW, DVD-RW, and DVD-RAM)

- Reads and writes are slower than with magnetic disk
- **Juke-box** systems, with large numbers of removable disks, a few drives, and a mechanism for automatic loading/unloading of disks available for storing large volumes of data

Magnetic Tapes

- non-volatile, used primarily for backup (to recover from disk failure), and for archival data
- **sequential-access** – much slower than disk
- very high capacity (40 to 300 GB tapes available)
- Hold large volumes of data and provide high transfer rates
  - Few GB for DAT (Digital Audio Tape) format, 10-40 GB with DLT (Digital Linear Tape) format, 100 GB+ with Ultrium format, and 330 GB with Ampex helical scan format
  - Transfer rates from few to 10s of MB/s
- Currently the cheapest storage medium
  - Tapes are cheap, but cost of drives is very high
- Very slow access time in comparison to magnetic disks and optical disks
  - Limited to sequential access.
  - Some formats (Accelis) provide faster seek (10s of seconds) at cost of lower capacity
- Used mainly for backup, for storage of infrequently used information, and as an off-line medium for transferring information from one system to another.
- Tape jukeboxes used for very large capacity storage
  - (terabyte (10^{12} bytes) to petabye (10^{15} bytes)

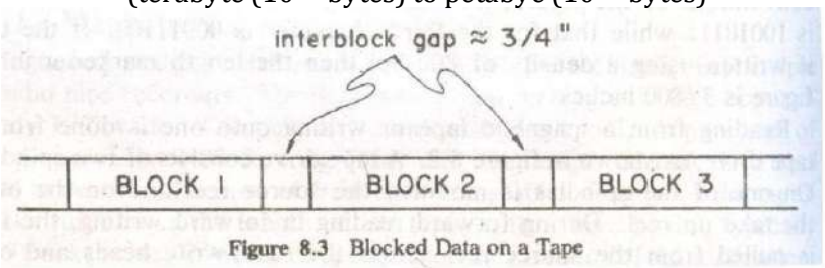


Figure 8.3 Blocked Data on a Tape

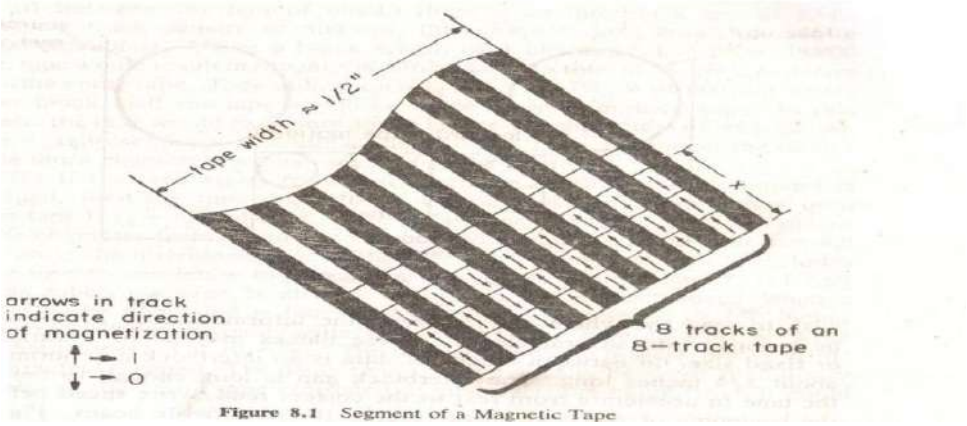


Figure 8.1 Segment of a Magnetic Tape

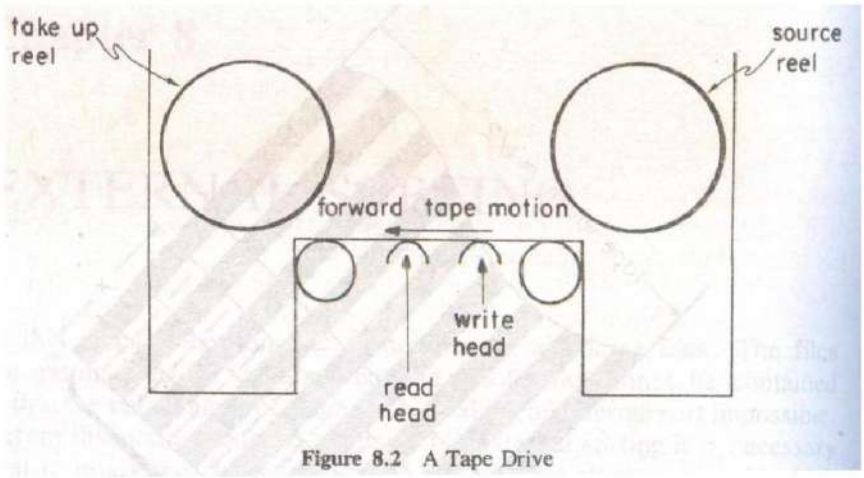


Figure 8.2 A Tape Drive

FLOPPY DISKS

- Mylar plastics usually 5 ½ or 8 inches in dia-coating magnetic material.
- 8 to 26 sector/track with 128 to 512 bytes.
- Capacity between 125 KB to 1 MB and transmission rate is 5 to 10 characters/m sec.

9. Explain sequential file organization / Explain in detail about indexing techniques (Nov 12, Nov 13,14)

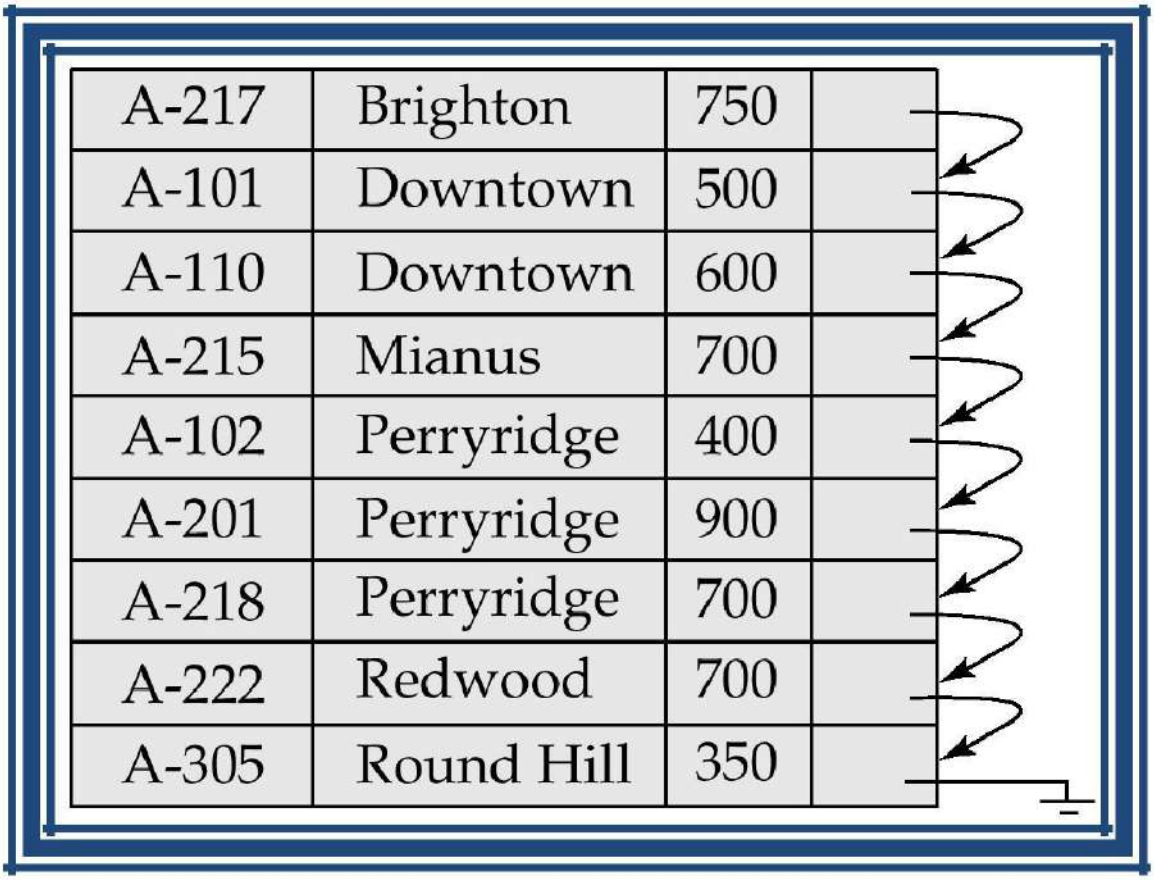
FILE ORGANIZATION

- The database is stored as a collection of *files*. Each file is a sequence of *records.* A record is a sequence of fields.
- Fields: Account_Number, Brach_Name and Balance.
- Collection of Records in a file is described by the following diagram,

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 2	A-215	Mianus	700
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600
record 8	A-218	Perryridge	700

SEQUENTIAL FILE ORGANIZATION

- Sequential files have data records stored in a specific order.
- Sequential file can be accessed in serially.
- Suitable for applications that require sequential processing of the entire file
- The records in the file are ordered by a search-key.
- A key is defined to the data records to uniquely identify each data.
- For e.g., in a Bank application the customer is uniquely identified by bank account number.
- The following figure shows how the records are organized by a **links or pointer** chains sequentially.



A-217	Brighton	750	
A-101	Downtown	500	
A-110	Downtown	600	
A-215	Mianus	700	
A-102	Perryridge	400	
A-201	Perryridge	900	
A-218	Perryridge	700	
A-222	Redwood	700	
A-305	Round Hill	350	

OPERATIONS PERFORMED IN SEQUENTIAL FILE ORGANIZATION

1. INSERTION OF RECORDS
2. UPDATION OF RECORDS
3. SEARCHING OF RECORDS
4. DELETION OF RECORDS

INSERTION AND DELETION IN SEQUENTIAL FILE ORGANIZATION

- Insertion and Deletion – use pointer chains
- Insertion –locate the position where the record is to be inserted
  - if there is free space insert there
  - if no free space, insert the record in an overflow block
  - In either case, pointer chain must be updated
- Need to reorganize the file from time to time to restore sequential order.

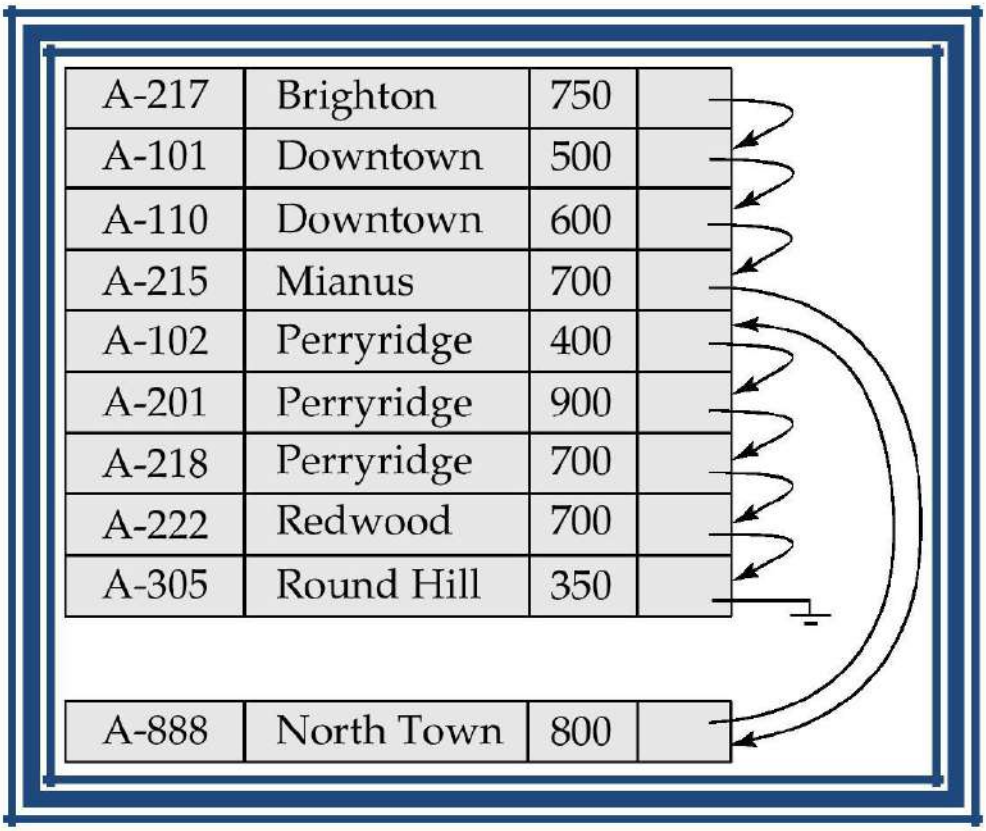


Fig. The role of Overflow block in sequential file organization

SEARCHING A RECORD IN A FILE

It involves looking or finding a record sequentially (one by one) in file until finding the required record.

INDEXED SEQUENTIAL FILE

- Each record of a file has a key field which uniquely identifies that record.
- An index consists of keys and address (physical disc locations)
- An index sequential file is a sequential file (i.e. sorted into order of a key field) which has an index.
- A full index to a file is one in which there is an entry for every record.
- Indexed sequential file are important for applications where data needs to be accessed sequentially and randomly using the index.
- An indexed sequential file allows fast access to a specific record.

E.g.:- A company may store details about its employees as an indexed sequential file.

Sometimes the file is accessed,

- Sequentially, for e.g.:- When the whole of the file is processed to produce pay slips at the end of the month.
- Randomly, may be an employee changes address, or a female employee gets married and changes her surname.

Disadvantage of Sequential Files - The retrieval of a record from a sequential file, on average, requires access to half the records in the file, making such enquiries not only inefficient but very time consuming for large files. To improve the query responses time of a sequential file, a type of indexing technique can be added.

Definition - An index is a set of y, address pairs. A sequential (or sorted on primary keys) file that is indexed is called an ***index sequential file***. The index provides for random access to records, while the sequential nature of the file provides easy access to the subsequent records as well as sequential processing.

STRUCTURE OF INDEX SEQUENTIAL FILES

An index-sequential file consists of the data plus one or more levels of indexes. When inserting a record, we have to maintain the sequence of records and this may necessitate shifting subsequent records.

For a large file this is a costly and inefficient process, instead, the records that overflow then logical area is shifted into a designated overflow area and a pointer is provided in the logical area are shifted into a designated overflow area and a pointer is provided in the logical area of ***associated index entry*** points to the overflow location.

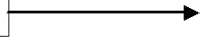
This is illustrated below (figure).

Record 615 is inserted in the original logical block causing a record to be moved to an overflow block.

611	612	614	618	624

Original logical block

611	612	614	615	618



Original logical block

overflow block

When records are forced into the overflow area as a result of insertion, the insertion process is simplified, but the search time is increased.

Deletion of records from index-sequential files creates logical gaps:-

- The records are not physically removed but only flagged as having been deleted.
- If there were a number of deletions, we may have a great amount of unused space.

ISAM (INDEX SEQUENTIAL ACCESS METHOD)

The important technique for building index based on the physical layout of the data in storage. When a record is stored by ISAM, its record key must be one of the fields in the record.

The records themselves are first sorted by record key into ascending order before they are sorted on one or more disk drives. ISAM will always maintain the records in this sorted order. Each record is stored on one of the tracks of disks

Since the tracks on a cylinder are labeled 0, 1, 2… The records that follow those on track 1 are placed on track 2. Track '0' is the next file cylinder. The cylinders are also labeled 0, 1, 2…

When ISAM retrieves a record, it needs to know,

- The cylinder,
- The track address, and

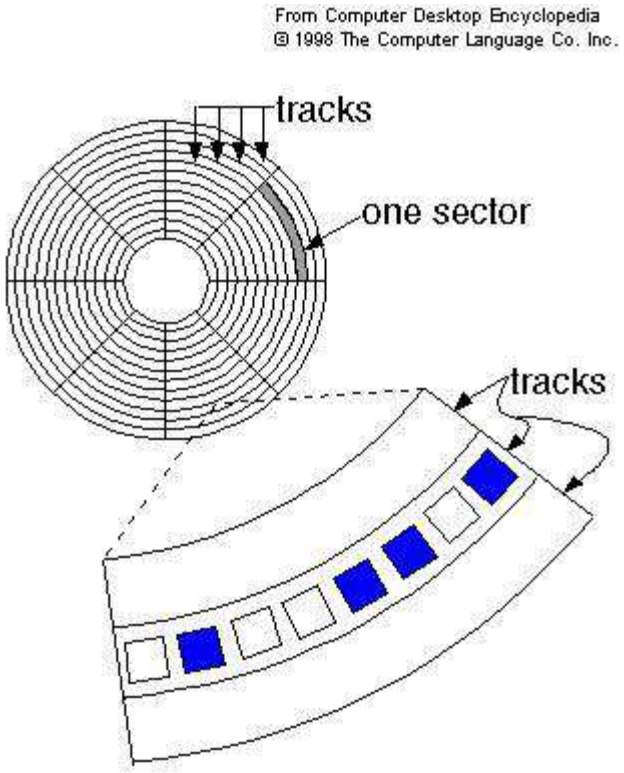
The record key.

When inserting a record, it is sufficient to know the largest record key on every track of the file.

For eg:- Suppose the largest key on the track 3 is 100 and the largest track 4 is 200. A record with key 175, if it exists in the file at all, must be on track 4. It can't be on track 3 as the largest key on that track is 100.

TRACK INDEX

Track index contains the largest key on every track and the hardware address of the track. Figure shows a typical track index for one cylinder of a file. In this cylinder, for e.g. 400 is shown to be the largest key on the track 3 and 700 the largest key on the cylinder.



1	150	2	200	3	400	……..	20	700

Track key track key track key ……..

Track Index

PROCEDURE TO FIND RECORD

First it positions the read/write mechanism over the appropriate cylinder and selects track 0.Suppose that the system seeks the key 350. The entry indicates the record, if it is to be found, will be on track 3. The read head for track 3 is selected and the rotation of the drive eventually brings the record with key 350. If it's under this read head. The fact that the index for this cylinder is on the cylinder itself means that no additional movement of the read/write mechanism is necessary.

3	400
Track	key

▪ ISAM keeps a cylinder index with an entry of each of its track indexes. Each entry in this index specifies the address of the every track index and the larges entry in each track index.

13	1650	14	1750	15	2000	16	3000	……
Cyl	key	cyl	key	cyl	key	cyl	key	

▪This cylinder index shows that on cylinder 15 the largest key that will be found is 2000. If ISAM is seeking a record 1880, an examination of cylinder 15 taking place. The read/write mechanism moves to cylinder 15, selects track 0, and consults the track index. Then track 1 is selected,and thus file is found out.

1	1800	2	1890	3	1990	……...
Track	key	track	key	track	key	

PRIME AREA

The file itself along with the track indexes is called the prime area.

OVERFLOW RECORDS IN ISAM

The records are forced into overflow area as a result of insertion. For examples, if the record at the end of a track are

……..	26	28	30	31	33	35	37

And record 34 is to be added, and then the track will be changed to,

……	26	28	30	31	33	34	35

And record 37 will be dropped off the end. The track's highest key is now 35 and the track index is changed accordingly. The question, of course, is what to do with record 37 that was dropped. If it is added to the next track, it will cause the record at the end of that track to be dropped at the end and a domino effect will cascade through all the records on the file.

In actual fact, there are two entries for each track on a given cylinder. We shall design them as 'N' and '0' entries, where "N" denotes a normal entry and "0" an overflow entry. Before overflow records are added to the file, both entries are the same. For eg:- the same track index for cylinder 6 of a file might appear as

N		0		N		0		N		
1	120	1	120	2	200	2	200	3	250	…….

In this case, both the N and 0 entries for track to designate that 200 is the largest key on this track. Suppose, in fact, that track 2 contains the following records.

130	145	150	……	180	190	200

As indicated by the track index, the largest key to be found on track 2 is 200. Now suppose record 185 is to be added to this track forcing record 200 off the end into the overflow area.

Track 2 now becomes,

130	145	150	……	130	185	190

As the largest key on track 2 is now 190, the N entry for this track in the index must be changed to 190 as follows.

N		0		N		0		N		
1	120	1	120	2	190	2	200	3	250	……

Suppose further that record 200 is placed in an overflow area on track 1 and is the first record on this overflow track. If this is designated as 10: 1, the overflow area should be changed as follows.

N		0		N		0		N			
1	120	1	120	2	190	10:1	200	3	250	……

In effect, then record 200 as become the first of many possible records in the overflow area.

If record 186 is added to track 2, forcing 190 off the end into the overflow area leaving track as

130	145	150	……	180	185	186

Then record 190 will be added as the second record in the overflow area, namely 10:2, and the overflow entry on the track index will be replaced by 10:2 so that the track index becomes

N		0		N		0		N		
1	120	1	120	2	180	10:2	200	3	250	……

Note that in the 0 entry the 200 is not changed as it still represents the largest record key in the overflow area. In fact the previous entry 10:1 is added to the latest record to be added to the overflow area, record 190, so that it is not lost the overflow area now looks like

#	200	10:1	190	……

The symbol; '#' indicates that record 200 doesn't point to another record. The overflow entry always contains two values.

One represents the largest key value in the overflow area that has been moved there from an individual track (200 in the above example) and the other contains the pointer to the smallest key in the overflow area(190 in the above example).

10. Describe about Multikey file organization

MULTIPLE KEY ACCESSES – MULTIKEY FILE ORGANISATION

Introduction:

- Multikey file access is a way to enable a single data file to support multiple access paths, each by a different key.
- This multiple key access has many real time applications in databases like employee details, hospital patient's details, student details to perform an effective search for complicated queries.
- A key which is used to access each record in a file is known as a *primary key*
- Usually the index or the serial number of a record is maintained as a primary key.
- All the other fields in the file are assumed as secondary index items (commonly known as secondary keys).
- These secondary keys are useful for handling queries based on the value of the items.

Let us consider hospital management system for our information retrieval as shown below

Primary Key	*Secondary Indexed Items*				
Hospitalization Number	*Patient's Name*	*Bed Number*	*Patient's Doctor*	*Drug Prescribed*	*M I*
0913628	Brown, J.H.	R67	Novak	XEN-02	
0931762	Copeck, J.A.	A02	Turtle	HYPOCH	
1013761	Rollie, R.K.	A04	Black	SULPH-3	
1029372	Cristie, L.H.	B21	Novak	CRYOL	
3056718	Jones, W.I.	R69	James	RESIN-A	
3084255	Watson, W.P.	B23	Turtle	CRYOL	
3931768	Andrews, A.K.	A09	Novak	NEOBEN	
4111234	McCord, G.A.	B24	James	HYPOCH	
4450902	Tash, R.R.	I33	King	SULPH-3	
6331313	Whyte, E.A.	A08	Black	LAX	
7614009	Dree, T.P.	R68	James	NEOBEN	
7729310	Bent, K.E.	A03	Novak	SULPH-3	

(*Bed number legend*: A = General Ward, B = Pediatric Ward, I = Intensive Care, M = Maternity Ward, R = Recovery Room.)

Fig (i)

In this file organization, we are going to discuss about the retrieval of the information's for the queries based on the secondary index items.

This multikey access can be done in 2 ways:

- Multilist (multiple threaded list) file organization
- Inverted file organization

MULTILIST FILE ORGANISATION:

In a multilist file organization, records which have equivalent values for a given secondary index items are linked together to form a list.

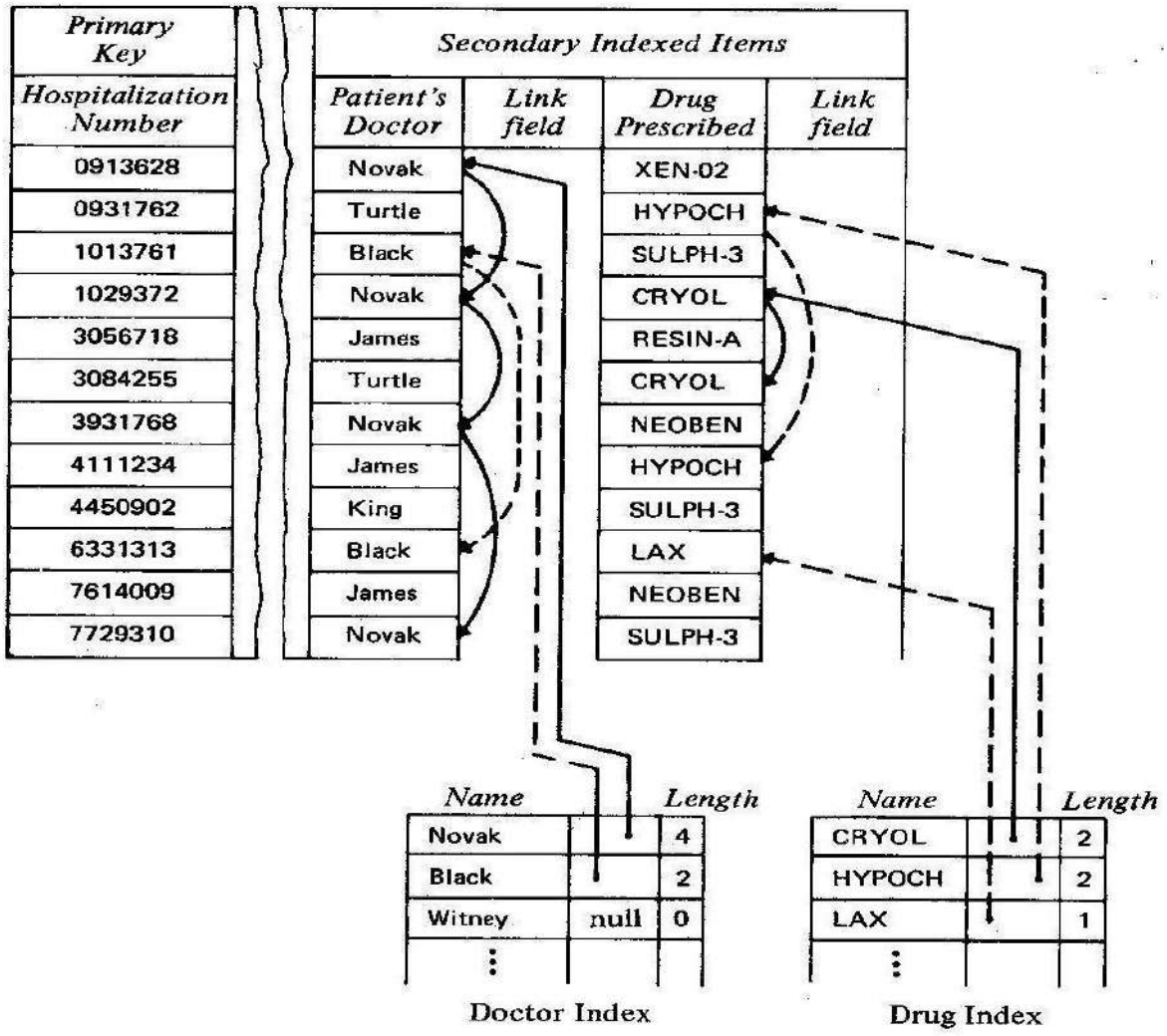


Fig (ii)

- ✓ The above figure illustrates two multilists – one for patient's doctor and another for drug prescribed.
- ✓ To provide a clear picture, lists for only three item values are shown, and a unique method is used for representing the links for each item in the diagram.
- ✓ In the above two multilists, each has 3 fields namely: name of the item value, link field and length of the list.

- ✓ In the figure (ii), the address field of an index element and the link field of the indexed item are depicted with arrows emanating from them.

Importance of length field:

- ✓ The length field in multilist is more useful in handling conjunctive queries.
- ✓ For illustration, consider a query: "Which patients of Dr. Novak require the drug CYROL?"
- ✓ We have to locate the patient's record with a doctor item of Novak and a drug of item CYROL
- ✓ In this case, the length field is used to detect the shorter list among the two.
- ✓ Obviously for this example, it is more efficient to retrieve the two records corresponding to patients taking CYROL and examine patient's doctor to Novak.

Memory allocation for multilist structure

Normally, these address fields contains absolute auxiliary memory or the primary key value. An auxiliary memory provides quicker access but it is affected by the physical movements of the records. The primary key value remains unaffected by the physical movement of the records but it is slower to access a record.

Primary Key	*Secondary Index Item*	
Hospitalization Number	*Patient's Doctor*	*Link Field*
0913628	Novak	1029372
0931762	Turtle	3084255
1013761	Black	6331313
1029372	Novak	3931768
3056718	James	4111234

Name	*Key*	*Length*
Novak	0913628	4
Black	1013761	2
Witney	null	0

Fig (iii)

The fig(iii)shows a primary key type of linkage for the doctor index using only first five records.

Advantages:

- ▪ Simplicity of programming and flexibility in performing updates.

Disadvantages:

- ▪ The greatest disadvantage of the multilist organization is to respond for a conjunctive query.
- ▪ All the records corresponding to the shortest list must be individually brought up into main memory for examination

11. Write about Inverted file organization in detail

INVERTED FILE ORGANISATION:

To overcome the disadvantages in multilist file organization , inverted file organization came into the view. Its main aim is to remove all the linkages from the file area and to place the list in a secondary index.

Patient's Name	*Primary Key*
Andrews, A.K.	3931768
Bent, K.E.	7729310
Brown, J.H.	0913628
Copeck, J.A.	0931762
Cristie, L.H.	1029372
Dree, T.P.	7614009
Jones, W.I.	3056718
McCord, G.A.	4111234
Rollie, R.K.	1013761
Tash, R.R.	4450902
Watson, W.P.	3084255
Whyte, F.A.	6331313

Fig (iv)

- ✓ The above figure shows the inverted list created for patient's name and hospitalization number.
- ✓ This inverted list provides an inverted relationship, that is, given a particular name, the hospitalization number can be located.
- ✓ This way allows a quick access in response to queries.

Ward	*Primary Key*
General(A)	0931762 1013761 3931768 6331313 7729310
Pediatric(B)	1029372 3084255 4111234
Intensive(I)	4450902
Maternity(M)	null
Recovery(R)	0913628 3056718 7614009

Fig (v)

- ✓ The above figure shows a partially inverted list of the hospitalization record and ward level only.
- ✓ This type of list is used to handle queries like "How many patients are in recovery?"
- ✓ An inverted list can appear as sequential, index sequential or direct file depending on the time to respond for a query.

Memory allocation in inverted file organization

If a list is not extremely long, it can be stored in the main memory itself. But for many large applications like employee details, hospital management, it is not possible.

Likewise, here in fig (iv), the patient's name list is very long and it cannot be stored in main memory for ready access. But in the case of fig(v), The patient's ward list has 5 sublists and so it can be stored and retrieved from the main memory itself.

Advantages:

- The major advantage in inverted list is its ability to handle queries with conjunctive term.
- The statistics concerning the number of times a secondary index item has been used can be easily kept.

Disadvantages:

- The secondary items being inverted generally have to be included in both inverted list and the master file

Comparison and Tradeoff in the Design of Multikey File

Both inverted files and multi-list files have

- An index for each secondary key.
- An index entry for each distinct value of the secondary key. In either file organization
- The index may be tabular or tree-structured.
- The entries in an index may or may not be sorted.
- The pointers to data records may be direct or indirect.

The indexes differ in that

- An entry in an inversion index has a pointer to each data record with that value.
- An entry in a multi-list index has a pointer to the first data record with that value.

Thus an inversion index may have variable-length entries whereas a multi-list index has fixed-length entries

12. Explain the concept of virtual memory

VIRTUAL MEMORY

- Some of the large programs cannot fit in main memory for execution. The usual solution is to introduce management schemes that are intelligently allocates portions of memory to users as necessary for the efficient running of their programs. The use of virtual memory is to achieve this goal.
- One type of system which provides logical extension is called as virtual memory system.
- When a program is executing and referencing data, all virtual addresses are translated automatically by the operating system into real main memory addresses
- There are three types of virtual memory system

(i) Paging

(ii) Segmentation

(iii) Paging with segmentation

Paging:

- Paging is a memory management technique in which virtual address space is split into fixed length blocks called pages.
- Main memory space is divided into physical sections of equal size subsection called page frames.
- The virtual address in a paging section is divided into two components 'p' and

  'd' p- Page number

  d- Page offset
- OS maintains a page table for each process.
- Page table shows the frame location for each page of the process
- If the bit is set invalid, the page will not be in the main memory.

Advantages of paging

- Paging eliminates fragmentation Segmentation
- Segmentation is a memory management scheme it divides the program into smaller blocks called segments.
- Segment can be defined as a logical grouping of information such as arrays or data area.
- Segmentation is a variable size
- Logical address using segmentation consist of two parts

  (i) Segment number(s)

  (ii) Offset(d)
- Segmentation eliminates internal fragmentation but suffers from external fragmentation.
- Segment table is used for mapping the two dimensional user defined addresses into an one dimensional physical addresses

Segment table consists of,

(i) Segment base(contains the starting physical address where the segment resides in main memory)

(ii) Segment limit(specifies the length of the segment)

Advantages of segmentation

Segmentation is visible to the user.

Difference between Paging and Segmentation:

	PAGING		**SEGMENTATION**
(I)	Divisions is performed by Os	(i)	Divisions are performed by users
(II)	It is made up of fixed block	(ii)	It is variable segments

Pages

(iii) Faster than segmentation (iii) Slower than paging

(iv) No external fragmentation (iv) Segmentation suffers from External fragmentation

13. What is VSAM? Explain in detail

VSAM FILES [VIRTUAL STORAGE ACCESS METHOD]

Disadvantages of index sequenced access method,

- The major disadvantage of index sequence access method is that the file grows performances decreases rapidly because of overflows.
- So there arises a need for periodic reorganization.
- But reorganization is an expensive process
- During reorganization , the files becomes unavailable

Virtual Storage Access Method

- VSAM was designed to replace all the access methods like,
  - Sequential Access Method
  - Index sequential Access Method
  - Direct Access Method
- VSAM files are made up of two components
  - Index & Data

The VSAM index and data are assigned to distinct block of Virtual storage called a control interval.

The control interval contains a number of empty index and data blocks, which are used when a data blocks overflows the index entry I1 indicates that the highest key value of the data block I2 is 73. The pointer to data blocks I2 is indicated by I2.

HANDLING OVERFLOW

- Suppose the records to be added have the key values of 55 & 60.These records will logically be added into data blocks I2.However, since I2 has a block size of 4.
- Only one record can be added without an overflow. The solution used in VSAM is to split the logical block i2 into two blocks, let us say I2 & D7.
- The records are inserted in the correct logical sequence.
- In VSAM, a Number of control intervals are grouped together into a control area.
- An index exists for each control area. control interval can be viewed as a track and a control area as a cylinder of the Index-Sequential Organization

Two types of VSAM file,

- Key sequenced file

- Entry sequenced file

KEY SEQUENCED FILE

Records which are ordered by keys are known as key sequenced file.

ENTRY SEQUENCED FILE

Records which are ordered by sequence of entry are known as entry sequenced file.

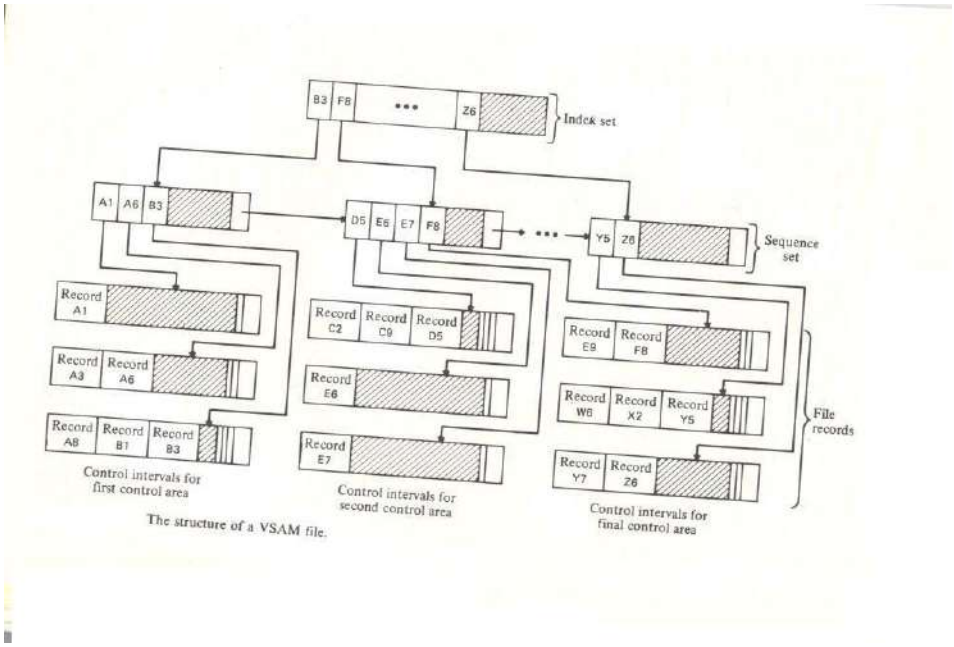
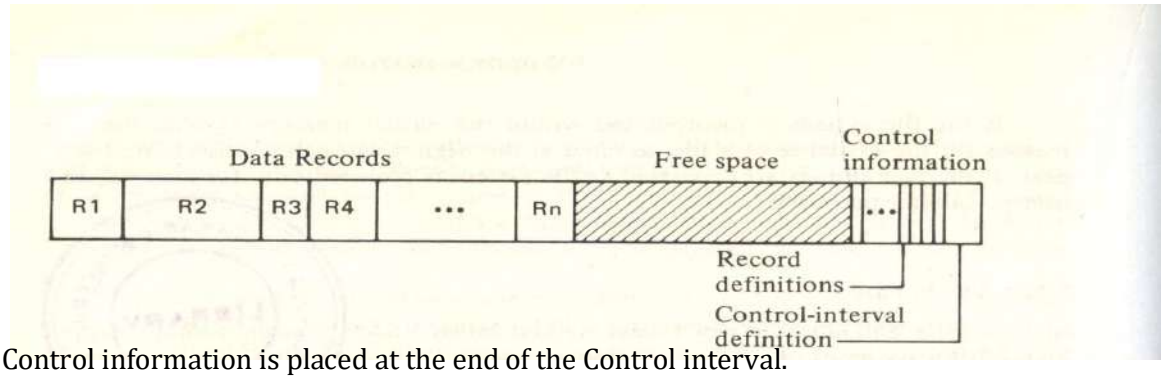


Fig.: PROCESS OF CONTROL INTERVAL IN KEY SEQUENCED FILE

CONTROL INTERVAL

Fig.: Logical Blocks of Control Interval



- .
- Control information is placed at the end of the Control interval.
- Record definition and Control interval definition are present in the Control Information.

CONTROL AREA

- Control interval are logically grouped together to form a Control Area.
- A set of indices is created for each control area and each particular set point to the control interval.

SEQUENCE SET

- A set of indices for all control area in a file is called Sequenced Set

INDEX SET

- Each index is contained in a record and set of all such index records is called index set. ENTRY OF INDEX SET AND SEQUENCED SET

- An entry in an index set record consist of highest key that an index record in the next lower level contains, together with a pointer to the lower level index record.

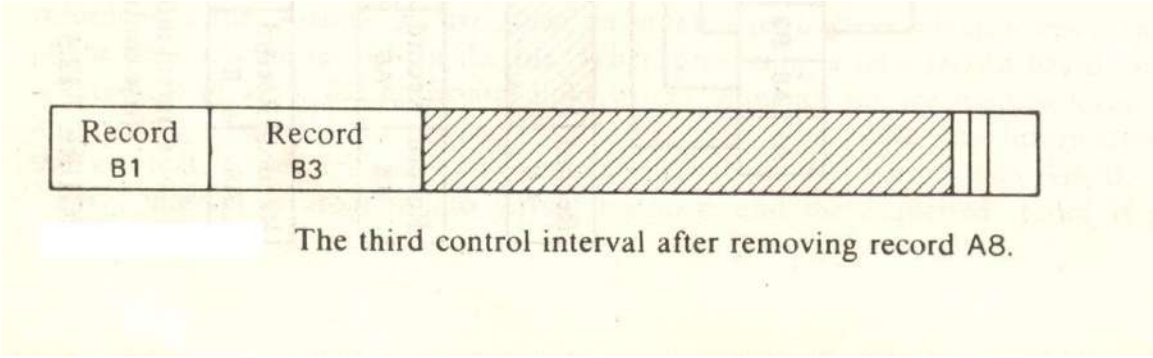
- For a sequence set record, an entry consists of the highest key in a control interval and a pointer to that control interval.

ADDITION AND DELETION OPERATION

DELETION OPERATION

- When records are deleted from a key sequenced file, the amount of space occupied by the record is recovered and added to the free space section of a control interval. This recovery of available space is accomplished by moving data records in the control interval to ensure that the data record section and free space section each remained contiguous areas.

  - The effect of removing the record with the key of A8 from the third control interval of the file as shown in the following figure.



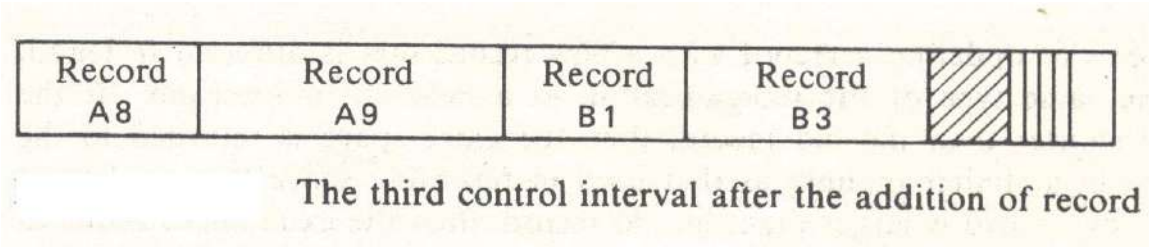
The third control interval after removing record A8.

  - Note that if record B3 is removed from this interval the third entry is the first sequence set record and first entry in the index set record must be altered to indicate that B1 is now the largest index in that particular control interval and control area.

ADDITION OPERATION

When a record is added to a keys sequenced file VSAM may move some of the existing records over to keep the record within the control interval physically in key sequence.

For eg.:- Suppose a record with key A9 is added to the third control interval as shown in the figure. The result of this insertion that the record B1 and B3 are moved, displaying some of the free space area



The third control interval after the addition of record

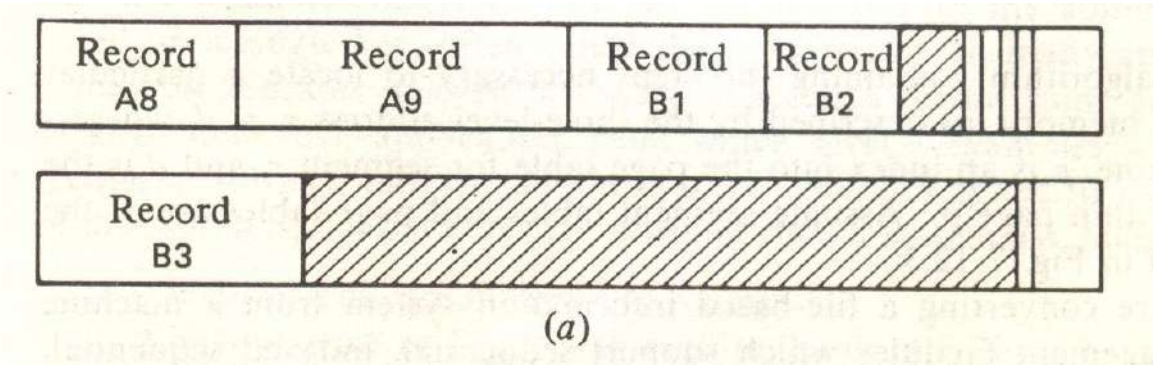
An obvious question arises.

What if there is not enough room in a control interval to accommodate the insertion of a new record?

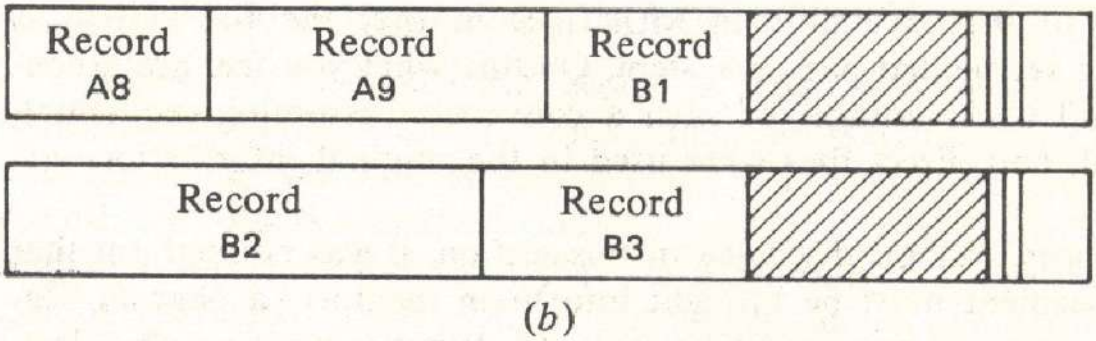
VSAM handles this situation by performing a control-interval split which is almost identical to a data-block split in a CDC scope indexed sequential file. In a control-interval split, stored records in the control interval are moved to an empty control interval in the same control area and the new record is inserted in its proper key sequence.

Just how the interval is split depends on the type of processing that is taking place .for a sequential insertion, the new record is placed in the new control interval, if possible, and all subsequent records are placed in the new control interval. Such a control interval split is as shown in the figure.

If the new record is too large to be placed to be placed in the new control interval, it and all remaining record in the original control interval are placed in the new control interval.



(a)



(*b*)

A control-interval split involving the insertion of (*a*) a small record B2 and (*b*) a large record B2.

14. Explain about tables & its types. (Apr 11, Nov 13)

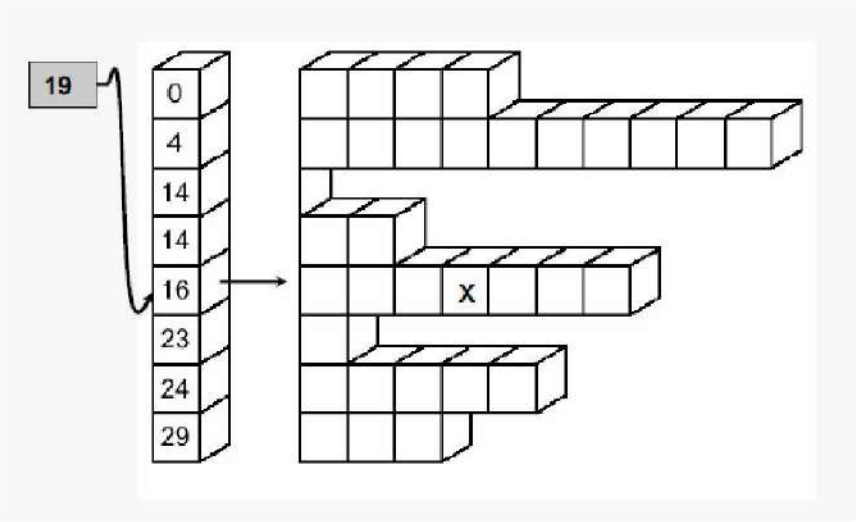
1.Table Lookup

- Theoretically, looking up an item by searching a list of N items and making key comparisons will, on average, require O(log N) work.
- As with sorting, we can "cheat" this result by organizing the data so that the search may be carried out with no (or very few) key comparisons.
- In essence, we will store the data elements in a structure, known as a table, and provide that table with an efficient indexing scheme. The table index will allow us to rapidly look up a key value and immediately find the location of the corresponding record in the table. The table will support random access, or at least approximate that, so given a location we can then find the record in constant or nearly constant time.

2. Built-in Tables: Arrays

- All general-purpose programming languages provide native support for arrays, mimicking mathematical matrices or vectors.
- Arrays provide constant time random access if an element's location is specified using the array index.
- The use of arrays is familiar, and inherently boring at this stage. What we seek is a generalization of the idea to provide similar capabilities for scenarios in which an in-memory array of cells is inappropriate or inadequate.
- It helps, however, to understand how the array index is transformed into an address in memory.

3.Jagged tables:



Given a (linear) index value, search the access array to find the appropriate table row…

… the access array cell will contain a "pointer" to the beginning of the corresponding table row.

The row can then be quickly searched for the desired entry.

In some cases, a table is naturally thought of as being composed of rows, but the rows contain wildly varying numbers of elements:

Updating the access array entries when insertions or deletions are performed in the table is simple but costs O(R) if the table has R rows.

4.Supporting Lookup on Multiple Key Values

In many cases, a table holding a collection of records must support efficient lookup by more than one key value. For example, we may have a set of customer records consisting of a name field, an address field and a phone number field. If we use a simple array, sorted by name, we have good support for searching on the name field, but not for address or phone number searches:

0	Baker, John S	17 King Street	2884285
1	Byers, Carolyn	118 Maple Street	4394231
2	Hill, Thomas M	39 King Street	2495723
3	Hill, Thomas M	High Towers #317	2829478
4	King, Barbara	High Towers #802	2863386
5	Moody, C L	High Towers #210	2822214
6	Roberts, L B	53 Ash Street	4372296

5.Inverted Tables

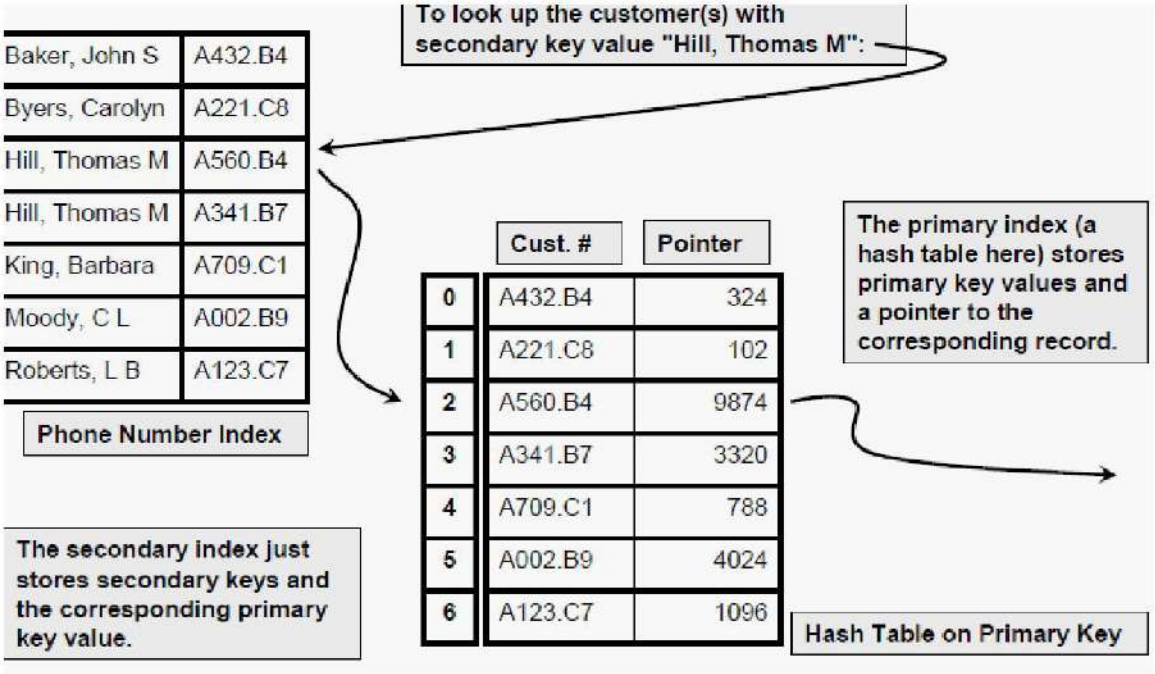
A solution is to provide an access array for each key value for which we need lookup capability:

2495723	2	
2822214	5	
2829478	3	
2863386	4	
2884285	0	
4372296	6	
4394231	1	

0	Baker, John S	17 King Street	2884285
1	Byers, Carolyn	118 Maple Street	4394231
2	Hill, Thomas M	39 King Street	2495723
3	Hill, Thomas M	High Towers #317	2829478
4	King, Barbara	High Towers #802	2863386
5	Moody, C L	High Towers #210	2822214
6	Roberts, L B	53 Ash Street	4372296

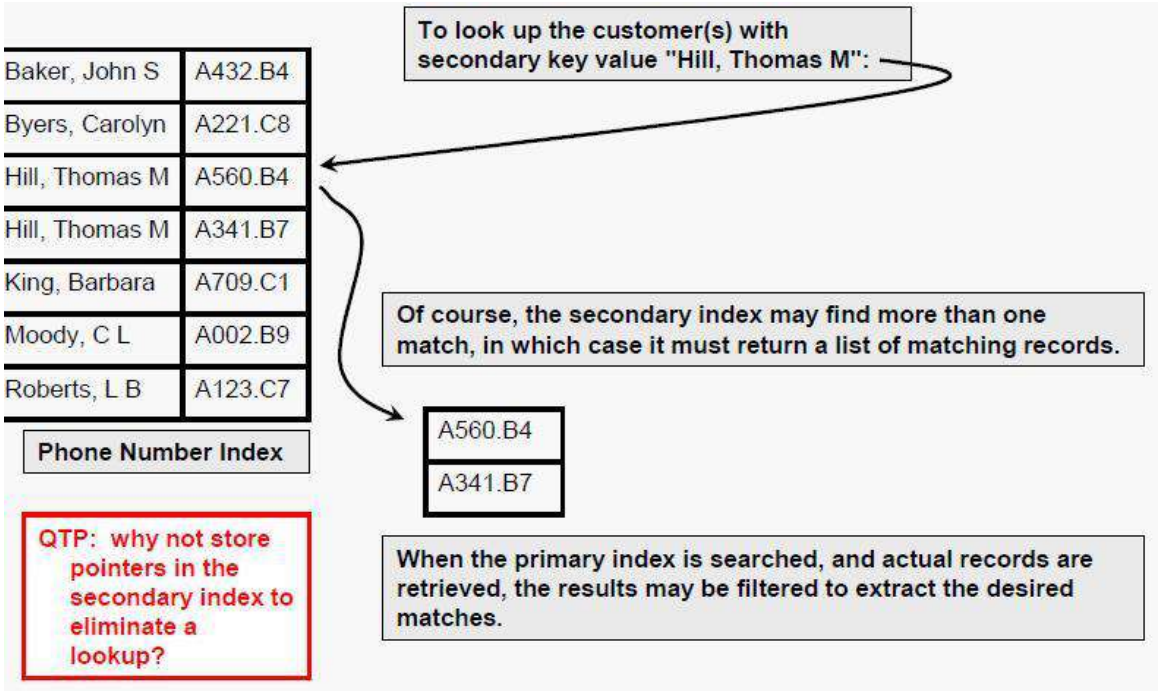
6.Inverted Table as a Secondary Index

A solution is to provide an access array for each key value for which we need lookup capability:



7.Inverted Table as a Secondary Index

A solution is to provide an access array for each key value for which we need lookup capability:



8.Table as an ADT

A table can be thought of as an abstract data type. Given a set of index values I, and a base type T, a table is a function M from I to T that supports the operations:

- access evaluate the function at any index value (retrieval)
- assignment modify the value of M(I) for any index value I
- creation define a new function M
- clearing remove all elements from I, so M's domain is empty

- insertion add a new value, x, to I and define M(x)
- deletion remove a value, x, from I (restricting M's domain)

Rectangular table:

Table Data

- Rectangular "table" data
- A very common structure for text or numeric data

As another example of how data is stored and manipulated in the computer, we'll look at "table data" -- a common a way to organize strings, numbers, dates in rectangular table structure. In particular, we'll start with data from the social security administration baby name site.

Social Security Baby Name Example

- Names for babies born each year
- Organized as a table
- **Fields:** name, rank, gender, year
- **Rows:** one row holds the data for one name

Table of baby-name data
(baby-2010.csv)

name	rank	gender	year
Jacob	1	boy	2010
Isabella	1	girl	2010
Ethan	2	boy	2010
Sophia	2	girl	2010
Michael	3	boy	2010

Field names

One row (4 fields)

2000 rows all told

- The **table** is made of 2000 **rows**, each row represents the data of one name
- Each row is divided into 4 **fields**
- Each of the 4 fields has its own name. The field names are: name, rank, gender, year

Tables Are Very Common

- Tables are a very common structure for computer data
- Number of fields is small (categories)
- Number of rows can be millions or billions
- e.g. email inbox: one row = one message, fields: date, subject, from, to, ...
- e.g. craigslist: one row = one thing for sale: description, price, seller, listing date, ...

Much of the information stored on computers uses this table structure. One "thing" we want to store -- a baby name, someone's contact info, a craigslist advertisement -- is one row. The number of fields that make up a row is fairly small -- essentially the fixed categories of information we think up for that sort of thing. For example one craigslist advertisement (stored in one row) has a few fields: a short description, a long description, a price, a seller, ... plus a few more fields.

The number of fields is small, but the number of rows can be quite large -- thousands or millions. When someone talks about a "database" on the computer, that builds on this basic idea of a table. Also storing data in a spreadsheet typically uses exactly this table structure.

PONDICHERRY UNIVERSITY QUESTIONS

11 MARKS

APRIL 2011 (ARREAR)

1. Explain about tables & its types. (Pg. No. 54) (Qn. No. 14)

NOV 2011(REGULAR)

1. Discuss about external storage devices. (Pg. No. 32) (Qn. No. 8)

MAY 2012(ARREAR)

1. What do you mean by hashing? Explain the various hashing functions (Pg. No. 18) (Qn. No. 3)

NOV 2012(REGULAR) NOV 2013(REGULAR)

1. Explain sequential file organization/Explain in detail about indexing techniques(Pg. No. 36)(Qn. No. 9)
2. Explain about tables & its types. (Pg. No. 54) (Qn. No. 14)
3. Describe about direct file organization (Pg. No. 22) (Qn. No. 5)

MAY 2014(ARREAR)

1. Explain in detail about symbol tables and hash tables. (Pg. No. 7) (Qn. No. 1)

NOV 2014(REGULAR)

1. Classify hashing functions and explain each with an example. (Pg. No. 18) (Qn. No. 3)
2. Explain the collision resolution techniques in hashing. (Pg. No. 18) (Qn. No. 3)
3. Explain in detail about sequential and direct file access. (Pg. No. 36) (Qn. No. 9) (Pg. No. 22) (Qn. No. 5)

MAY 2015(ARREAR)

1. **Define hash function and explain its methods in detail. (Pg. No. 18) (Qn. No. 3)**

Nov 2015(REGULAR)

1. **Explain Static tree Tables.**
2. **Briefly explain Sorting Performed ontapes and diskes.**

MAY 2016

1. **Write detailed notes on sequential file organization.**
2. **Explain any two external sorting techniques.**